

Computer Vision

Convolutional neural networks

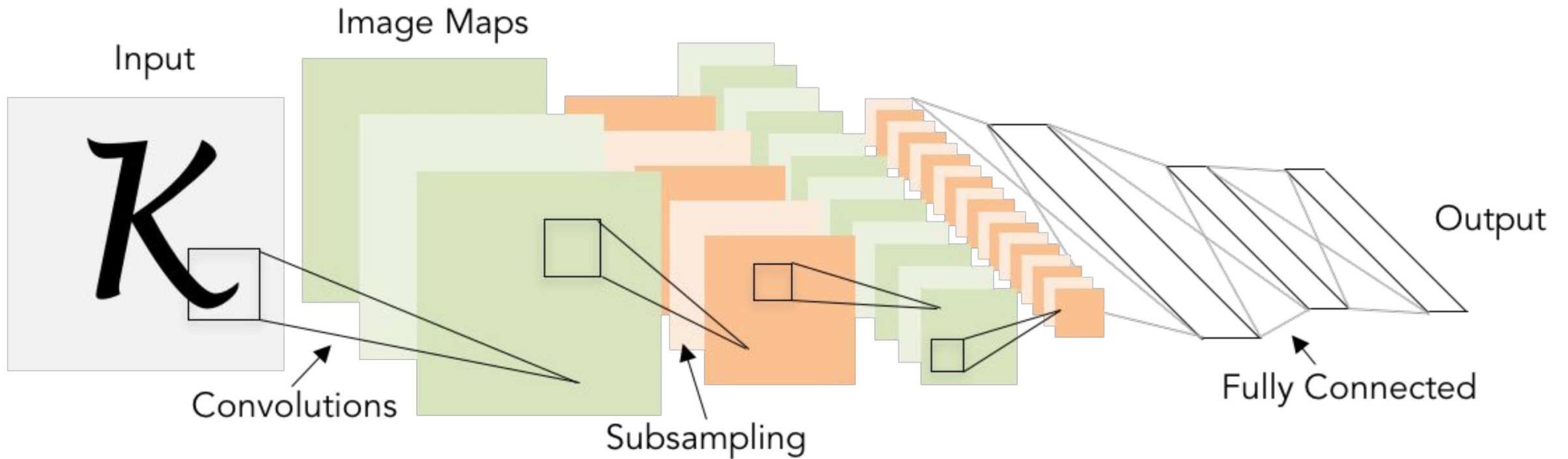
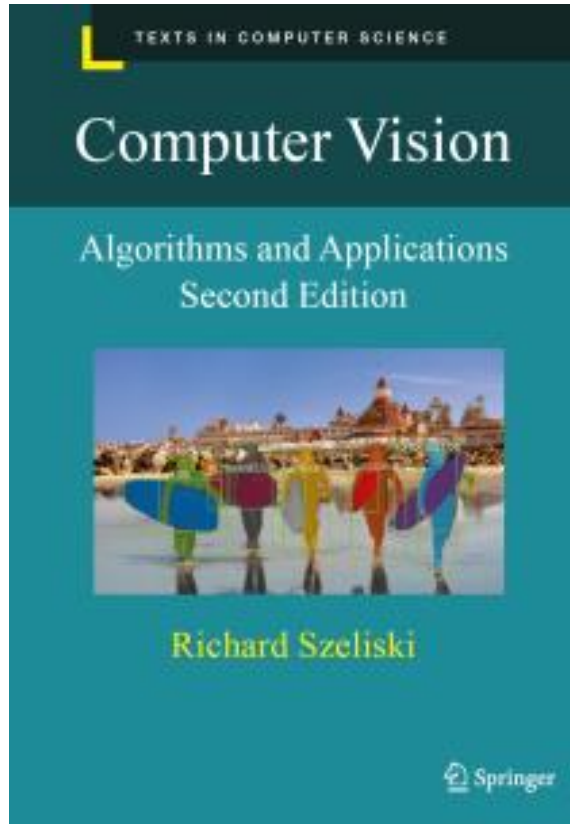


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

Important information



Textbook

Rick Szeliski, *Computer Vision: Algorithms and Applications* online at: <http://szeliski.org/Book/>

Many of the slides in this course are modified from the excellent class notes of similar courses offered in other schools by Noah Snavely, Prof Yung-Yu Chuang, Fredo Durand, Alyosha Efros, Bill Freeman, James Hays, Svetlana Lazebnik, Andrej Karpathy, Fei-Fei Li, Srinivasa Narasimhan, Silvio Savarese, Steve Seitz, Richard Szeliski, and Li Zhang. The instructor is extremely thankful to the researchers for making their notes available online. Please feel free to use and modify any of the slides, but acknowledge the original sources where appropriate.

All readings are from Richard Szeliski, *Computer Vision: Algorithms and Applications*, 2nd Edition, unless otherwise noted.

Readings

- Neural networks
 - <http://cs231n.github.io/neural-networks-1/>
 - <http://cs231n.github.io/neural-networks-2/>
 - <http://cs231n.github.io/neural-networks-3/>
 - <http://cs231n.github.io/neural-networks-case-study/>
- Convolutional neural networks
 - <http://cs231n.github.io/convolutional-networks/>

A bit of history...

The **Mark I Perceptron** machine was the first implementation of the perceptron algorithm.

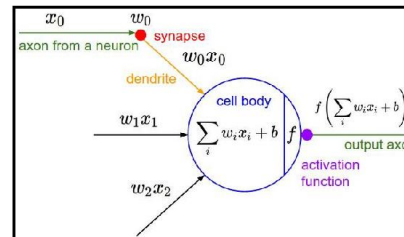
The machine was connected to a camera that used 20×20 cadmium sulfide photocells to produce a 400-pixel image.

recognized
letters of the alphabet

update rule:

$$w_i(t+1) = w_i(t) + \alpha(d_j - y_j(t))x_{j,i},$$

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$



Frank Rosenblatt, ~1957: Perceptron
(Cornell University)



[This image](#) by Rocky Acosta is licensed under [CC-BY 3.0](#)

A bit of history...

[Hinton and Salakhutdinov 2006]

Reinvigorated research in Deep Learning

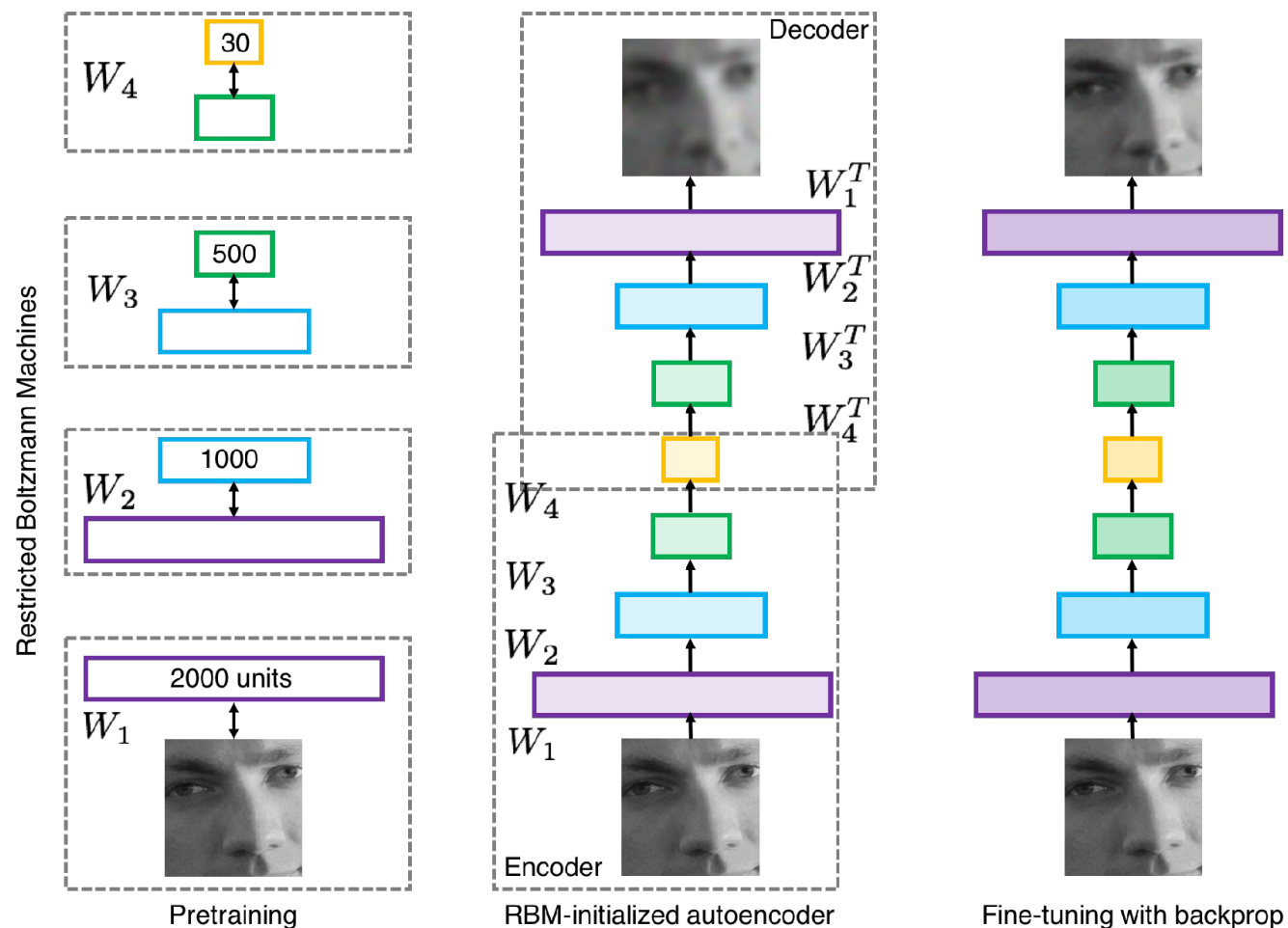


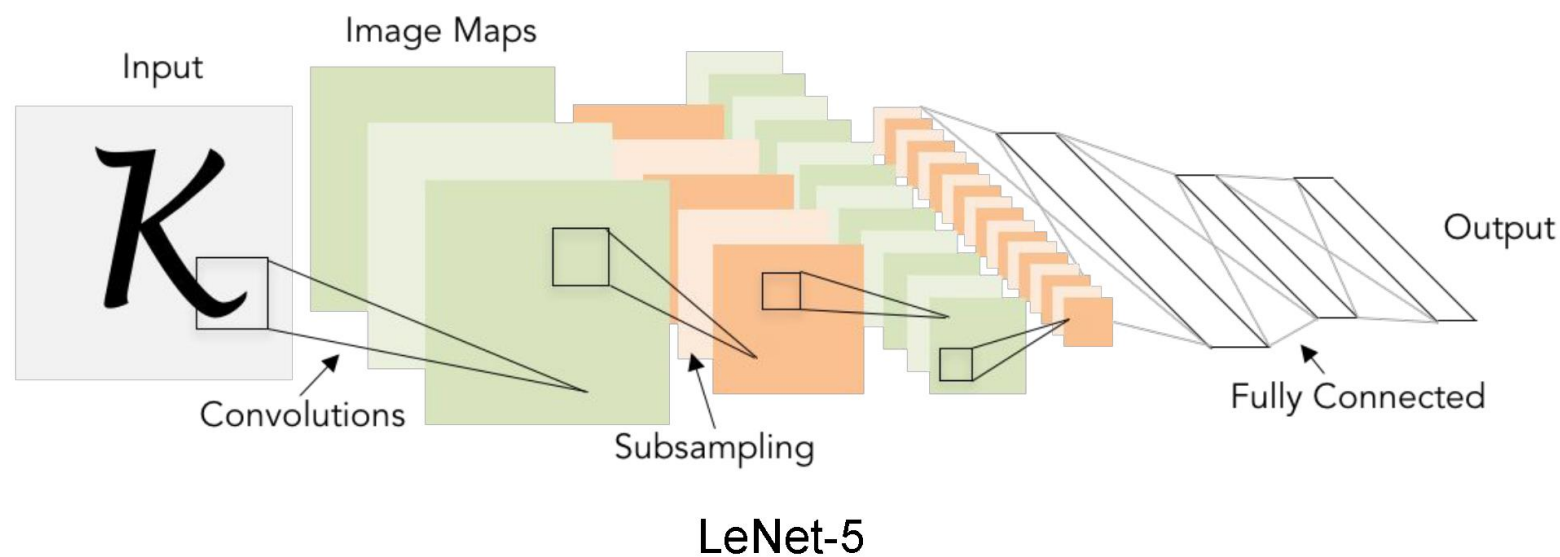
Illustration of Hinton and Salakhutdinov 2006 by Lane McIntosh, copyright CS231n 2017

Hinton and Salakhutdinov. Reducing the Dimensionality of Data with Neural Networks. *Science*, 2016.

A bit of history:

Gradient-based learning applied to document recognition

[LeCun, Bottou, Bengio, Haffner 1998]



A bit of history:

ImageNet Classification with Deep Convolutional Neural Networks

[Krizhevsky, Sutskever, Hinton, 2012]

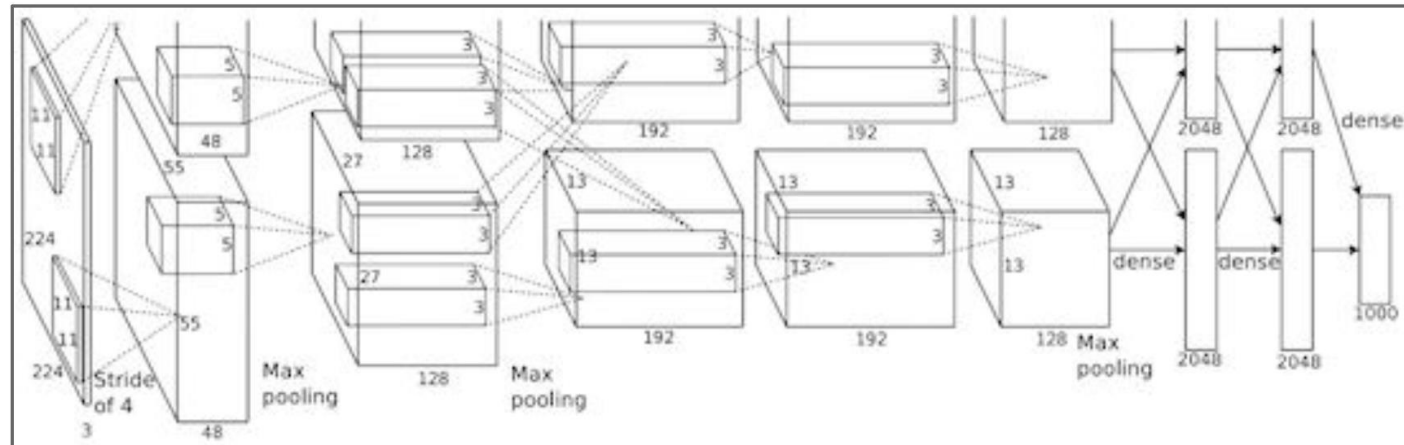
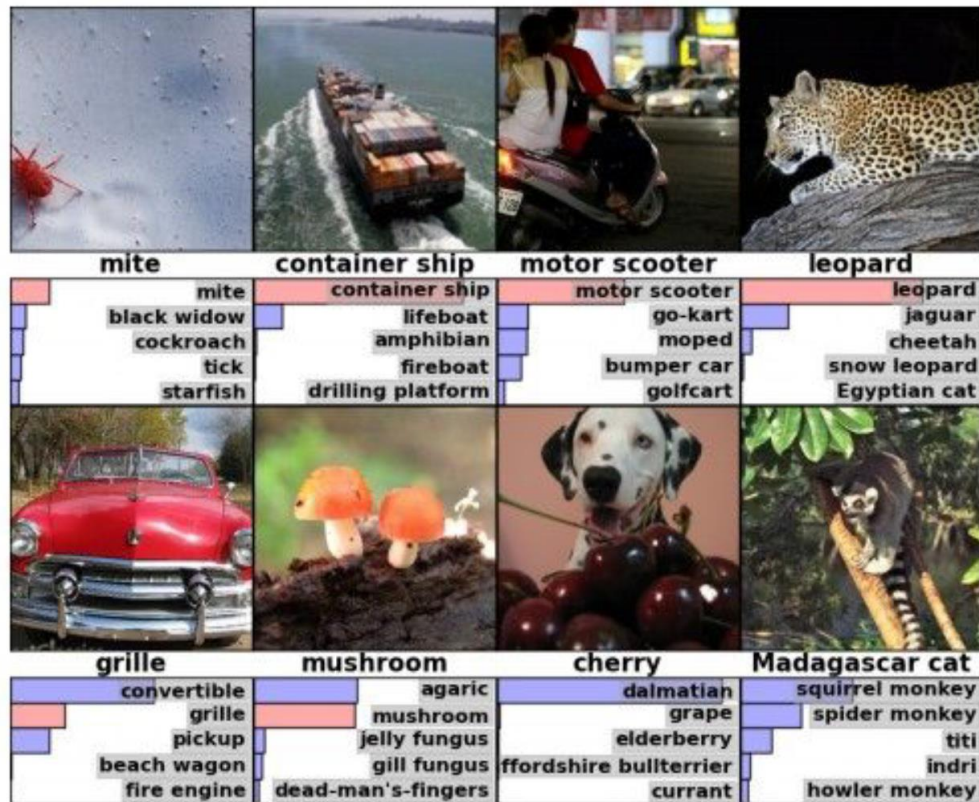


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

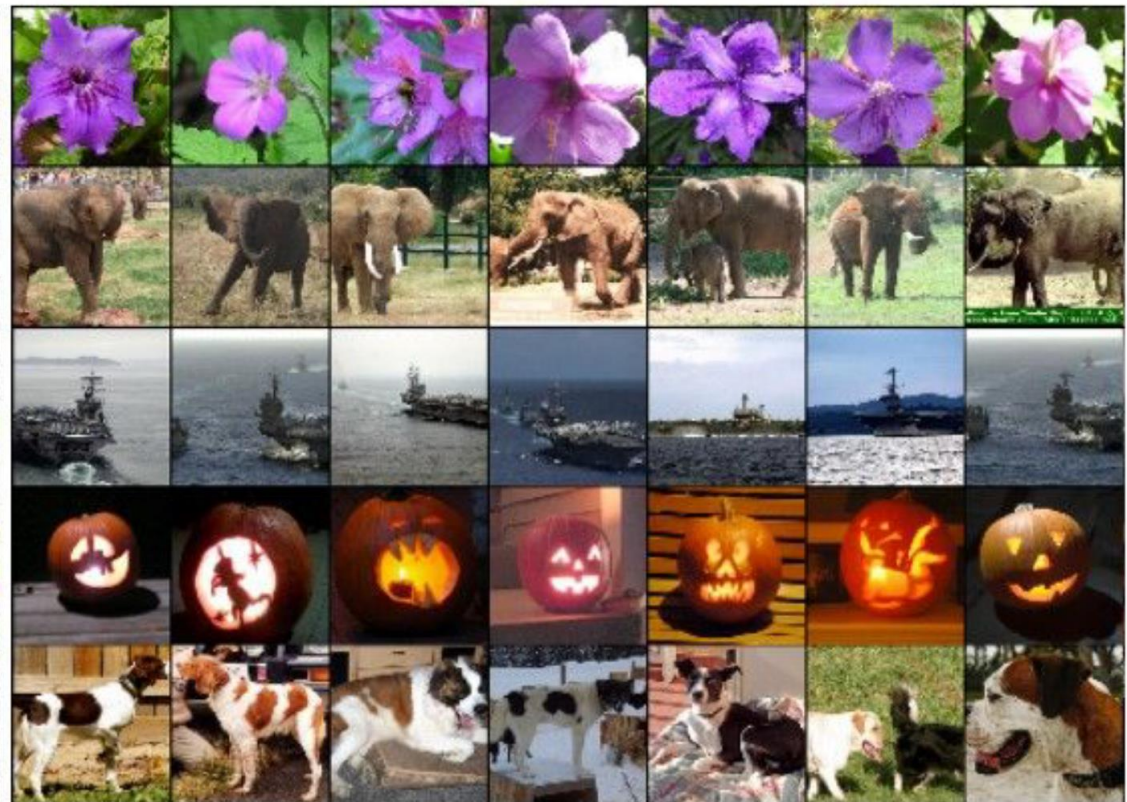
“AlexNet”

Fast-forward to today: ConvNets* are everywhere

Classification



Retrieval



Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

* and other recent architectures, like
Transformers

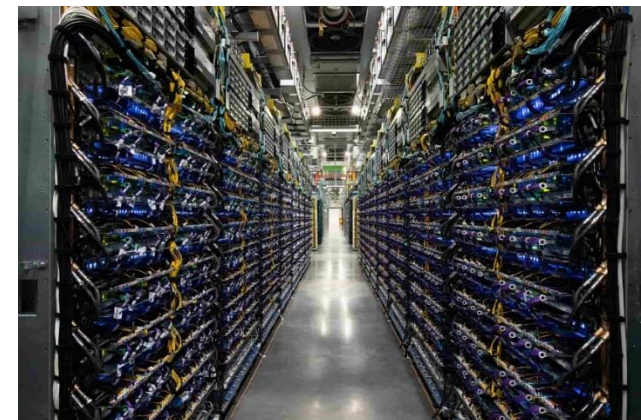
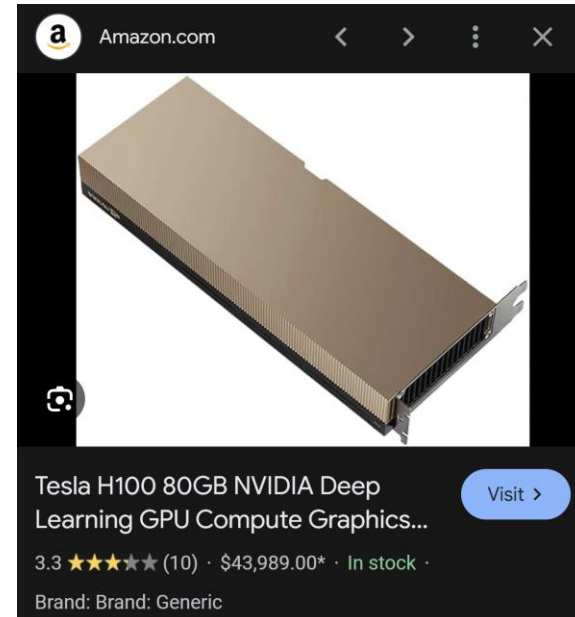
Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leu

Fast-forward to today: ConvNets are everywhere



Self-driving cars (video courtesy Tesla)

<https://www.tesla.com/AI>



Cloud TPU v4 Pods

<https://cloud.google.com/tpu/>

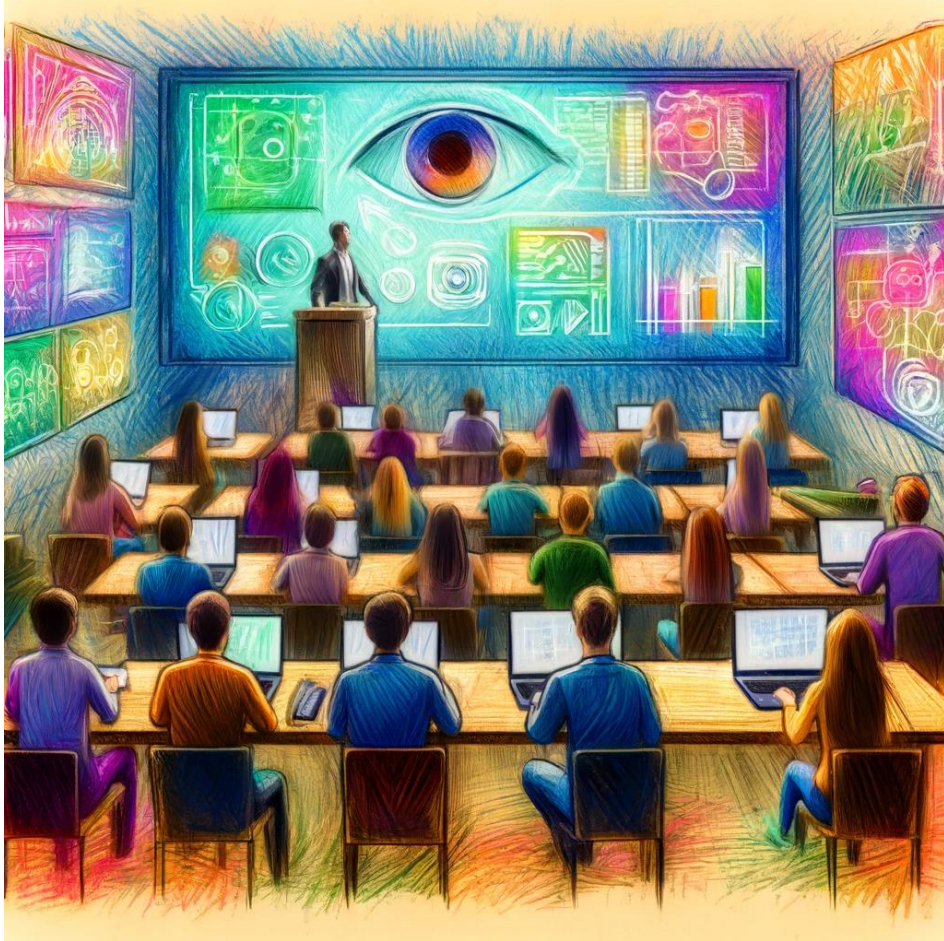
Text-to-image

An astronaut Teddy bears A
bowl of soup

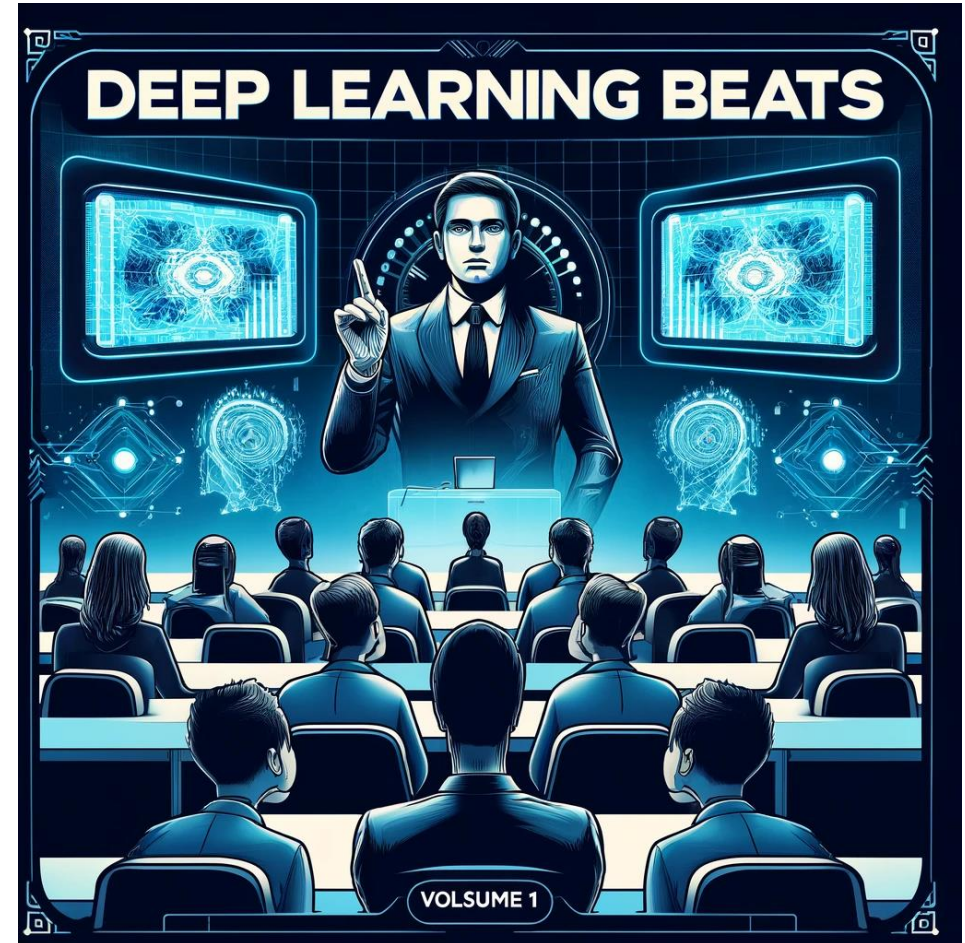
riding a horse lounging in a
tropical resort in space playing
basketball with cats in space

in a photorealistic style in the
style of Andy Warhol as a pencil
drawing





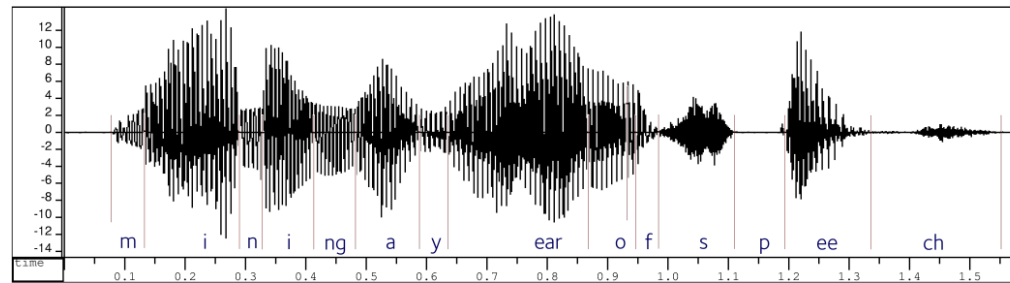
"A computer vision class watching a cool lecture, crayon drawing"



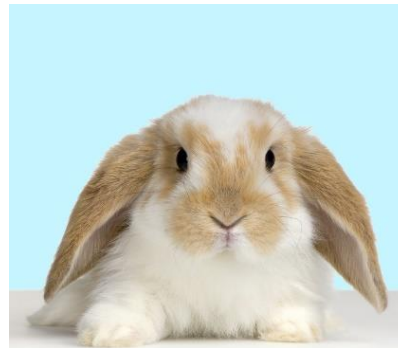
"A computer vision class watching a cool lecture, album cover"

What is a ConvNet?

- Version of deep neural networks designed for signals
 - 1D signals (e.g., speech waveforms)

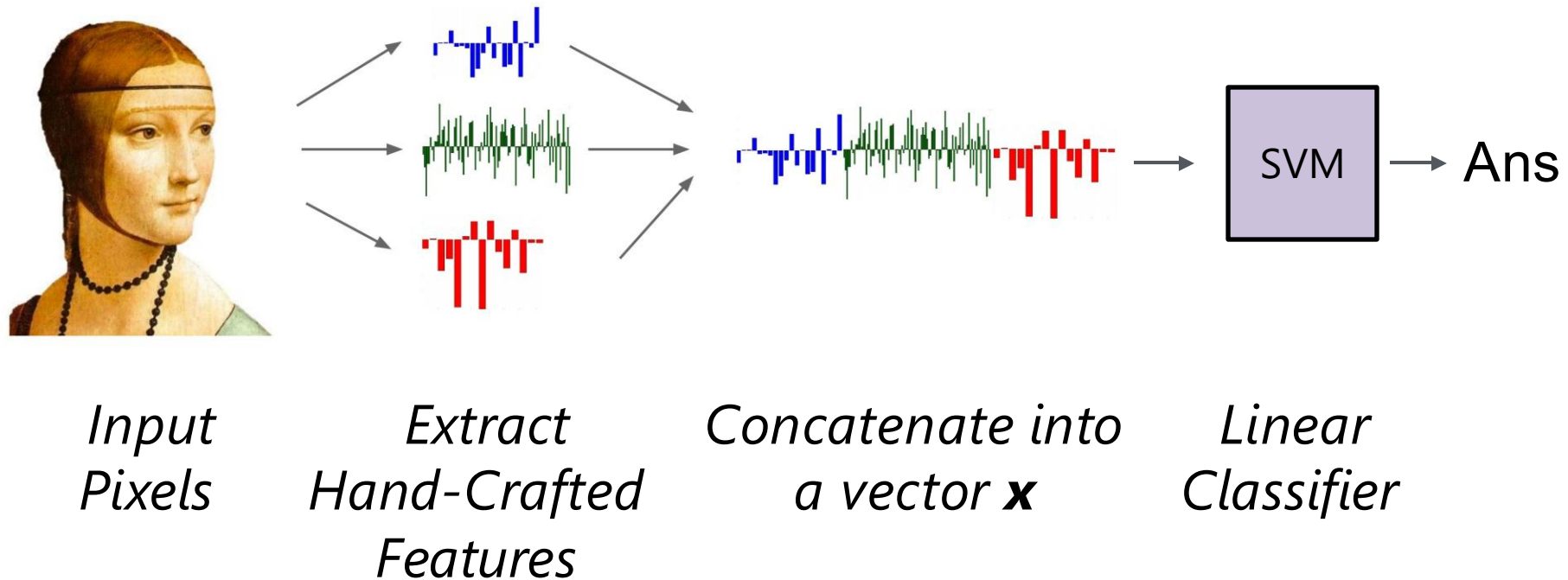


- 2D signals (e.g., images)

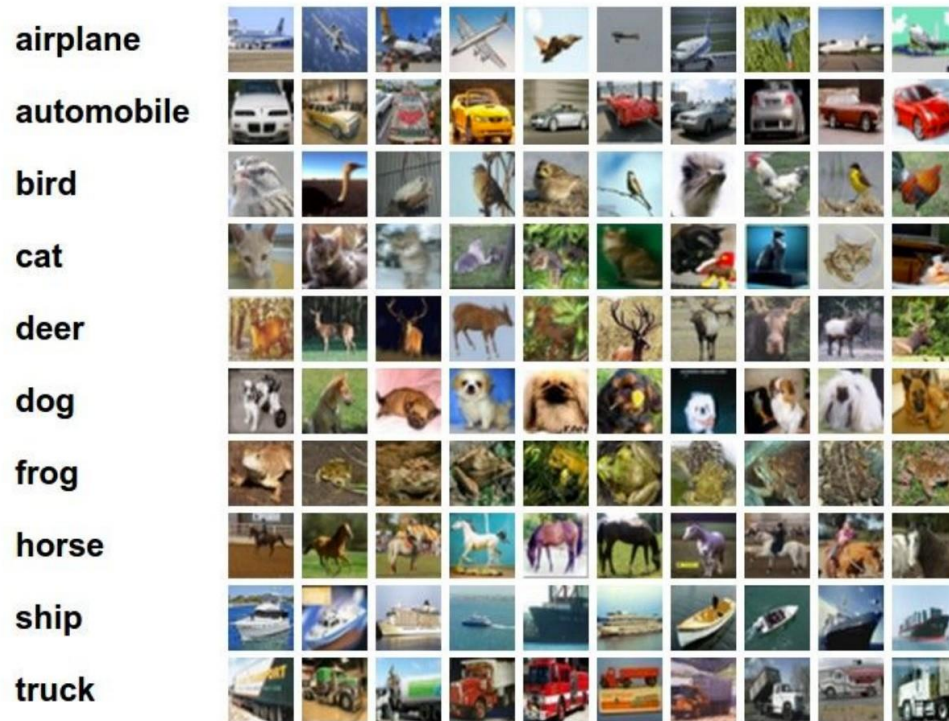


Motivation – Feature Learning

Life Before Deep Learning



Why use features? Why not pixels?

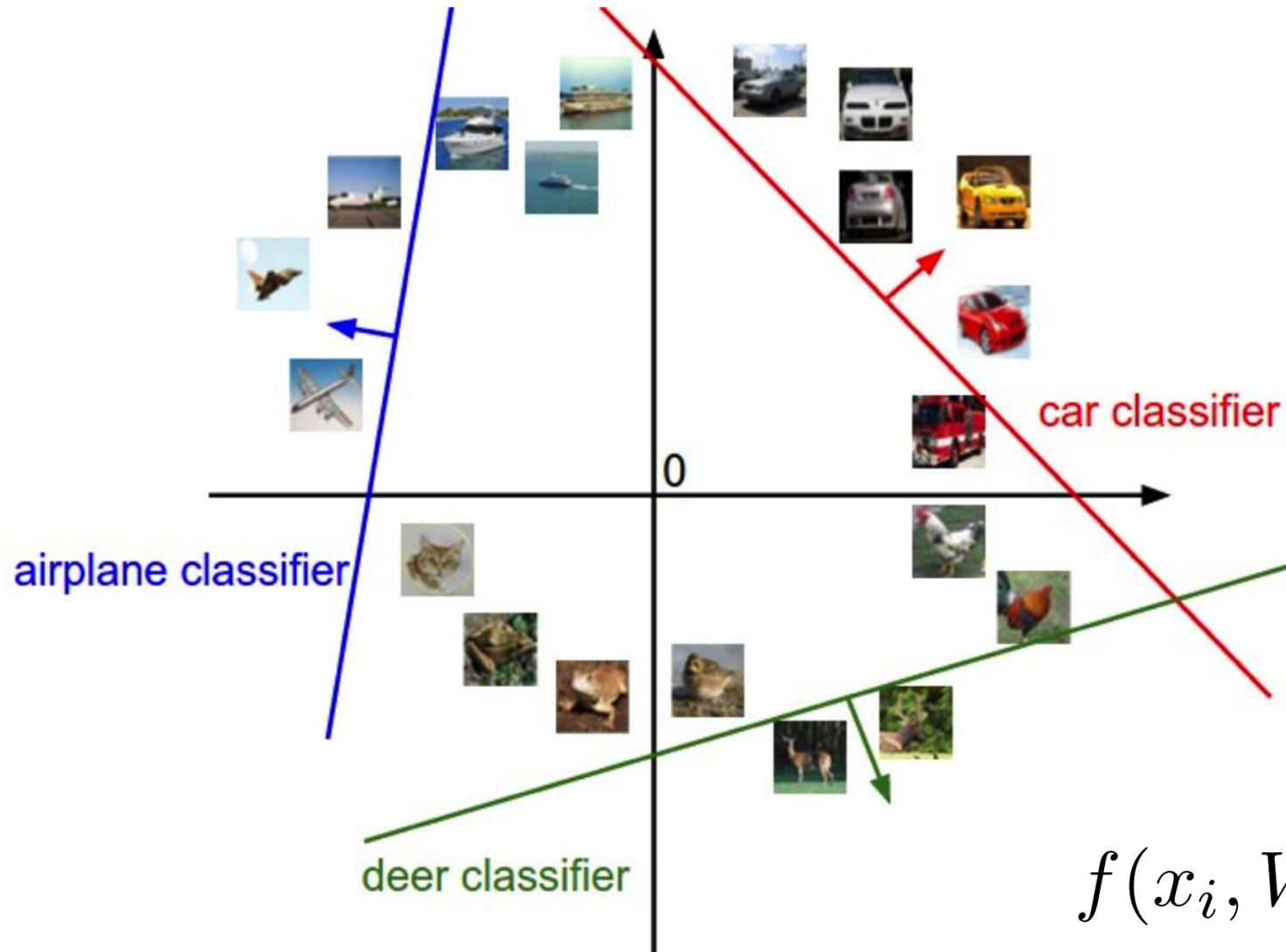


$$f(x_i, W) = Wx_i + b$$

Q: What would be a very hard set of classes for a linear classifier to distinguish?

(assuming x = pixels)

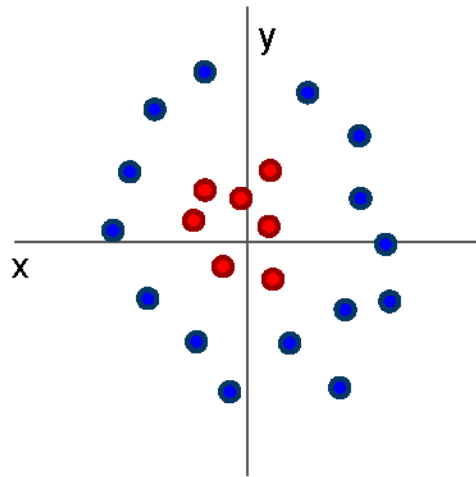
Goal: linearly separable classes



Aside: Image Features

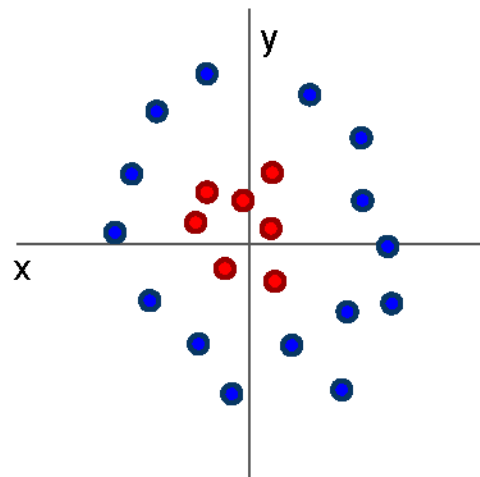


Image Features: Motivation



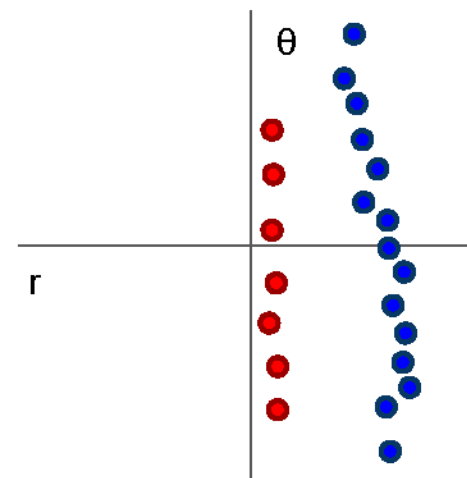
Cannot separate red
and blue points with
linear classifier

Image Features: Motivation



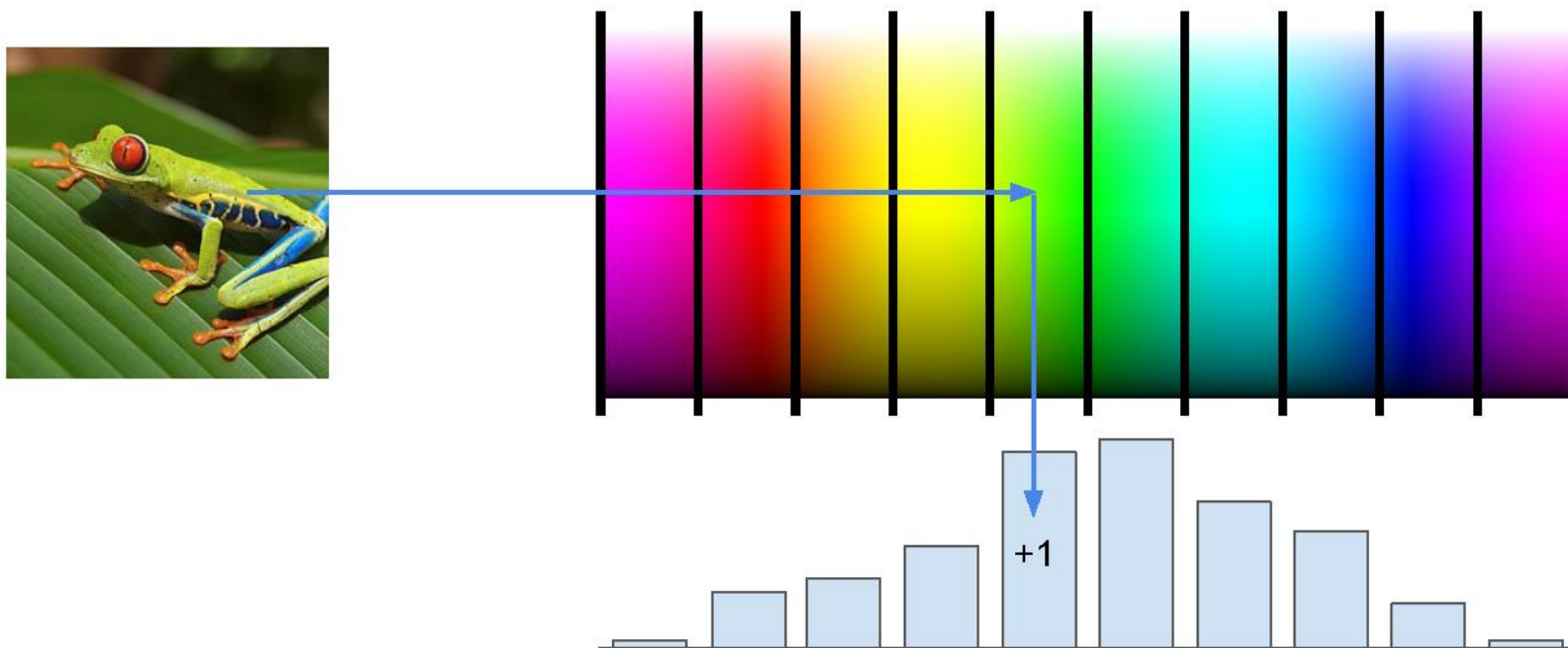
Cannot separate red
and blue points with
linear classifier

$$f(x, y) = (r(x, y), \theta(x, y))$$



After applying feature
transform, points can
be separated by linear
classifier

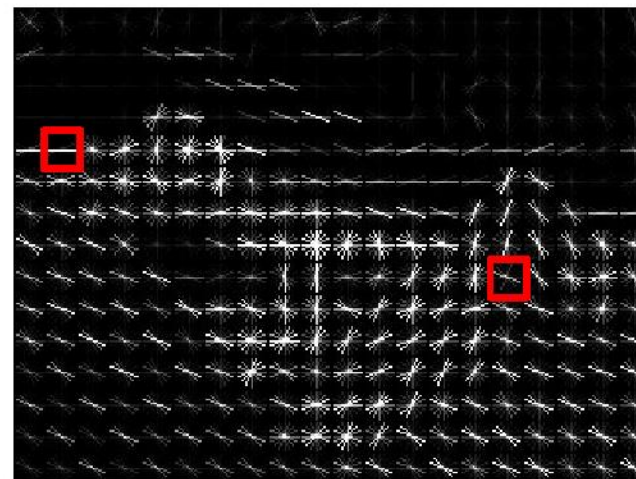
Example: Color Histogram



Example: Histogram of Oriented Gradients (HoG)



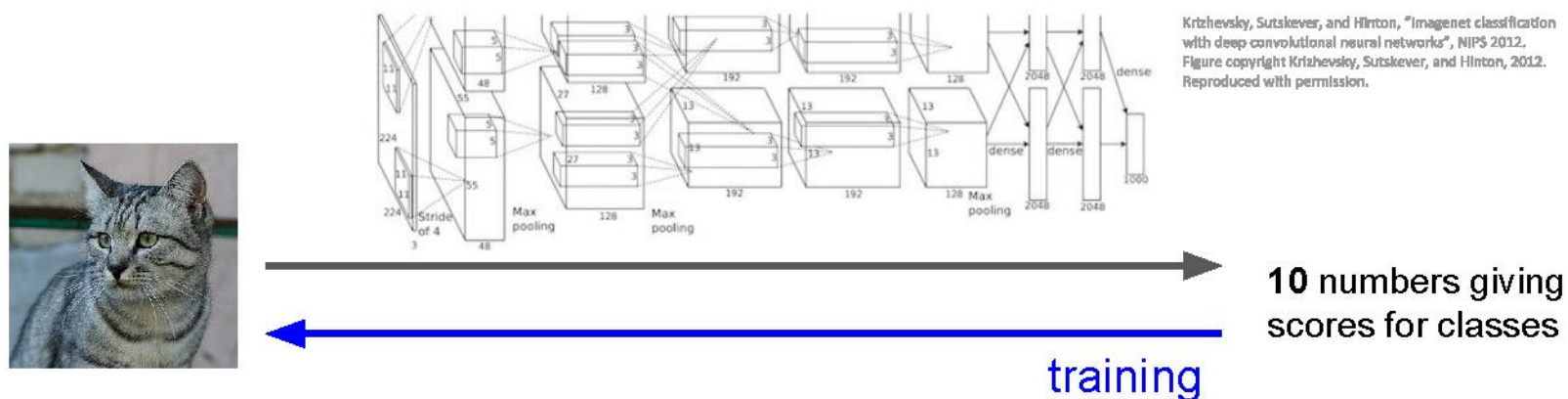
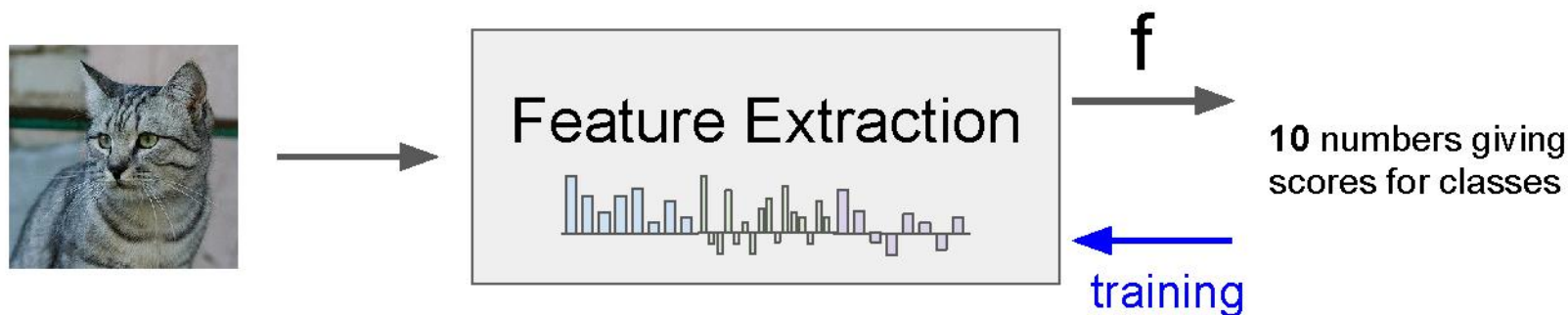
Divide image into 8x8 pixel regions
Within each region quantize edge
direction into 9 bins



Example: 320x240 image gets divided
into 40x30 bins; in each bin there are
9 numbers so feature vector has
 $30 \times 40 \times 9 = 10,800$ numbers

Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

Image features vs ConvNets

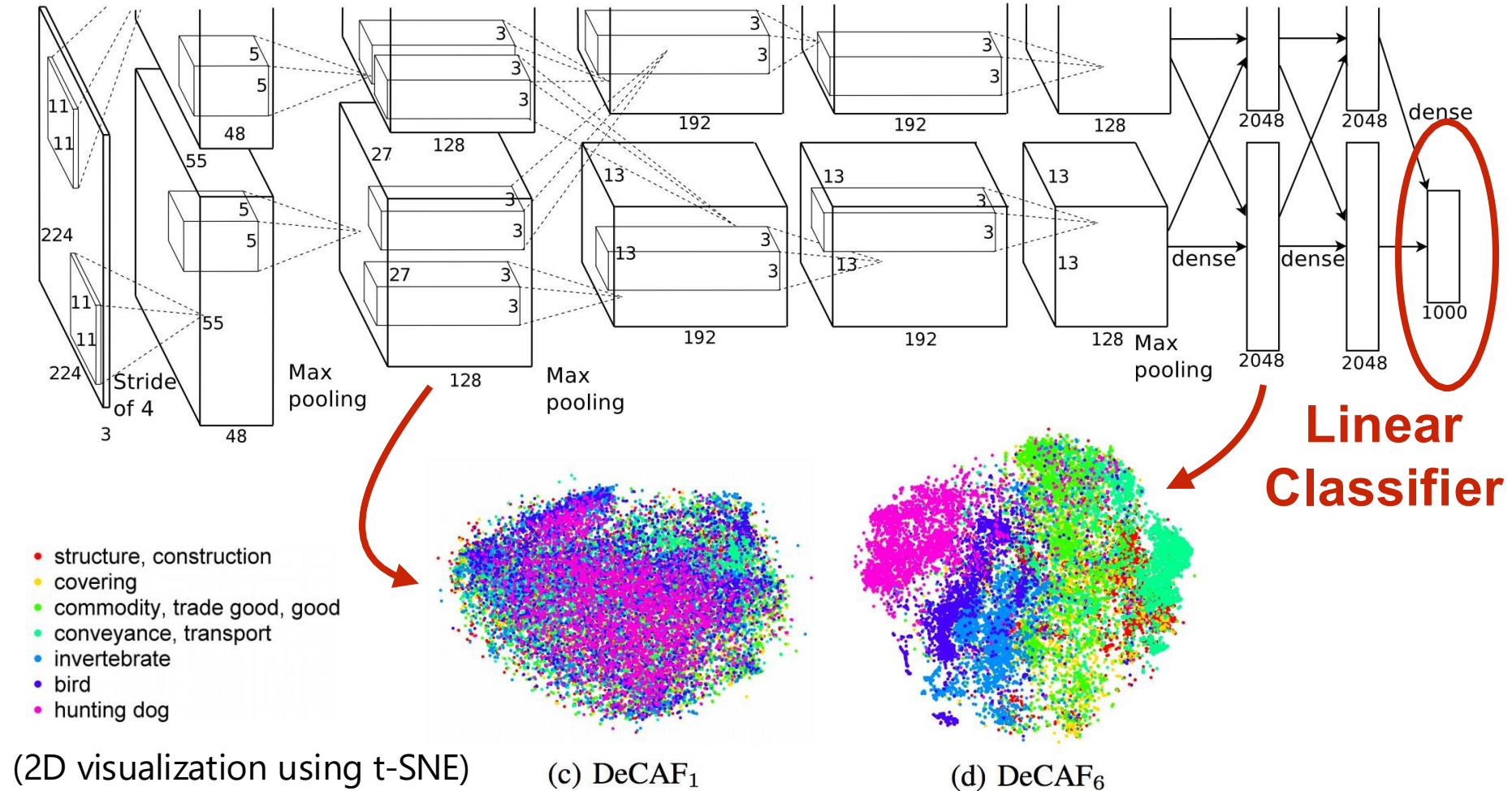


Last layer of many CNNs is a linear classifier



Key: perform enough processing so that by the time you get to the end of the network, the classes are linearly separable

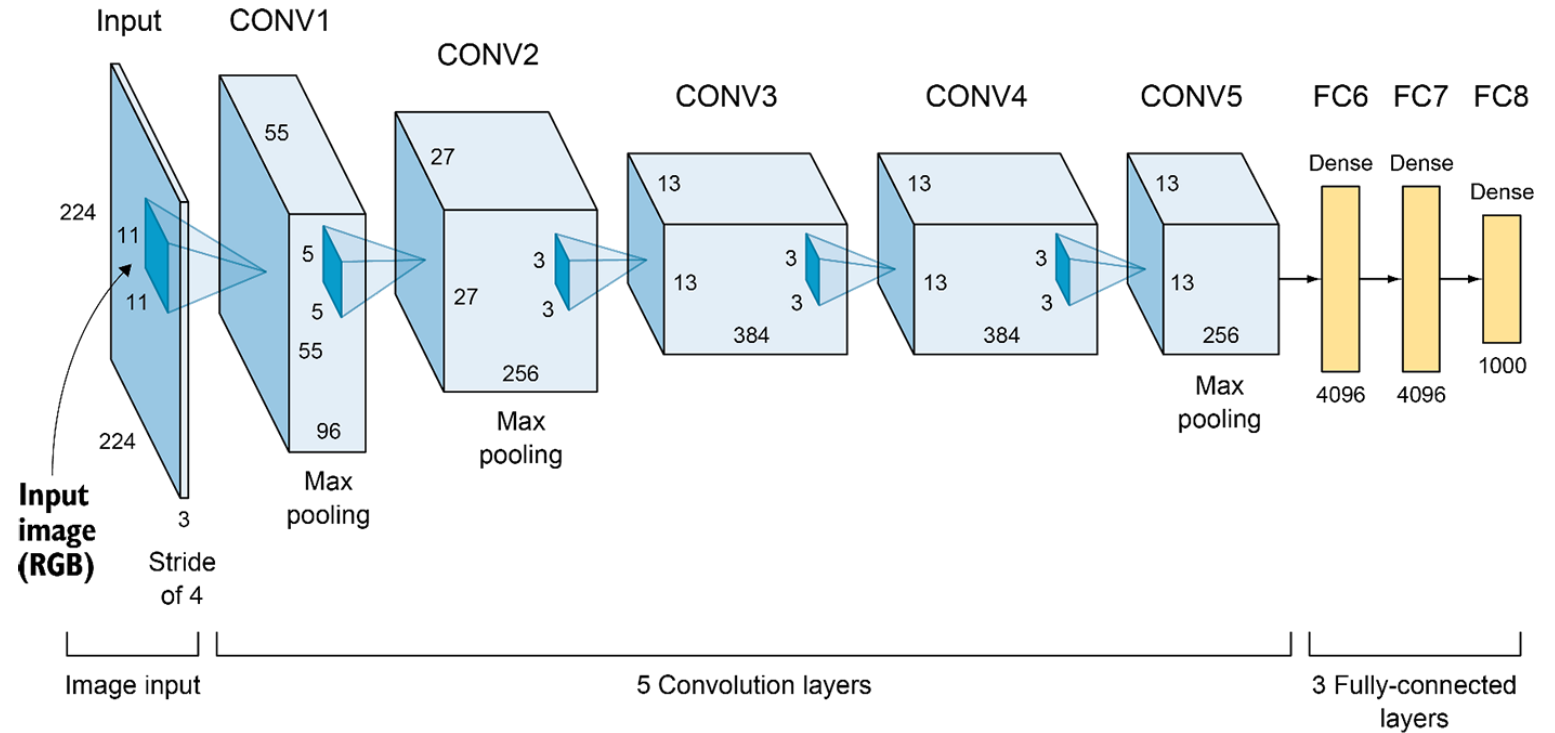
Visualizing AlexNet in 2D with t-SNE



Convolutional neural networks

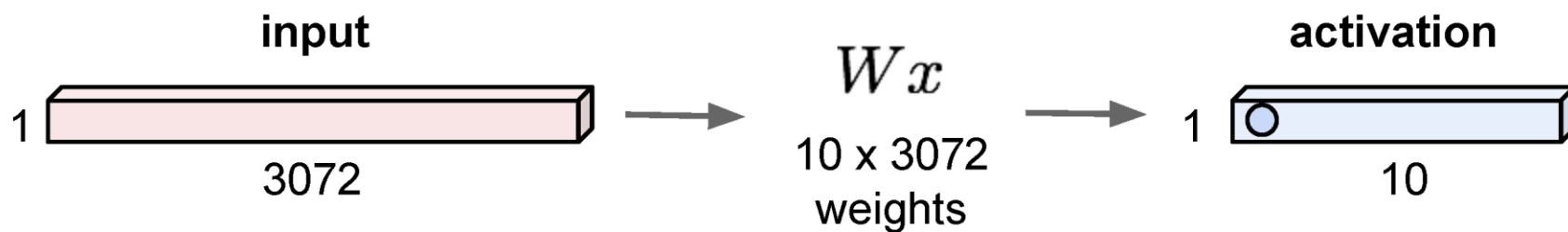
Layer types:

- *Convolutional layer*
- Pooling layer
- Fully-connected layer



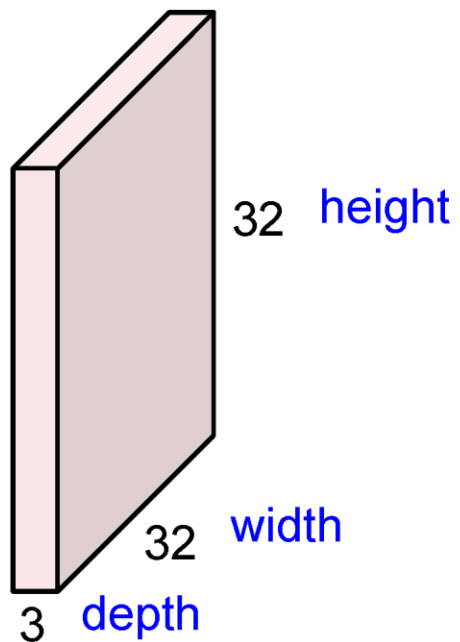
Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



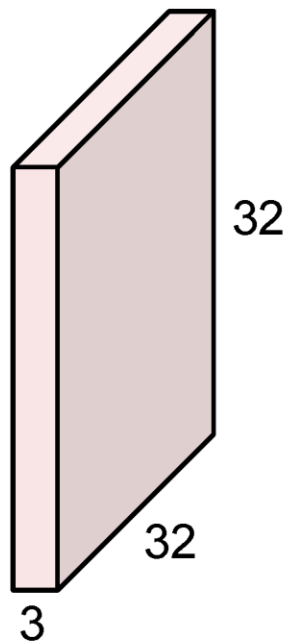
Convolution Layer

32x32x3 image -> preserve spatial structure



Convolution Layer

32x32x3 image



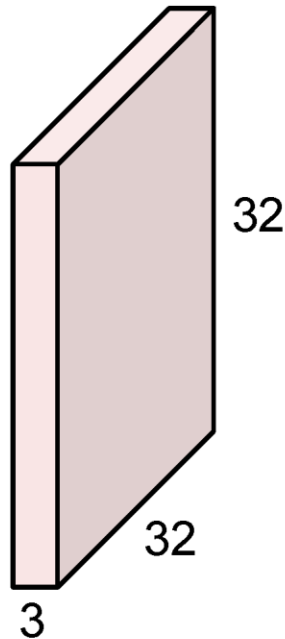
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

32x32x3 image



Filters always extend the full depth of the input volume

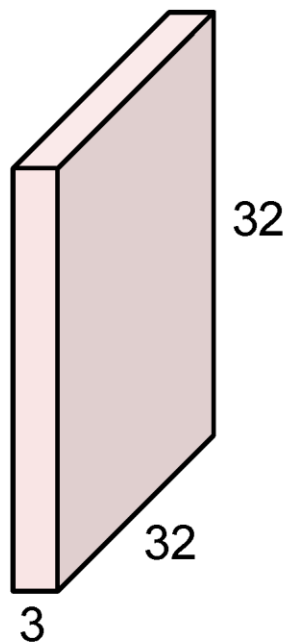
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

32x32x3 image



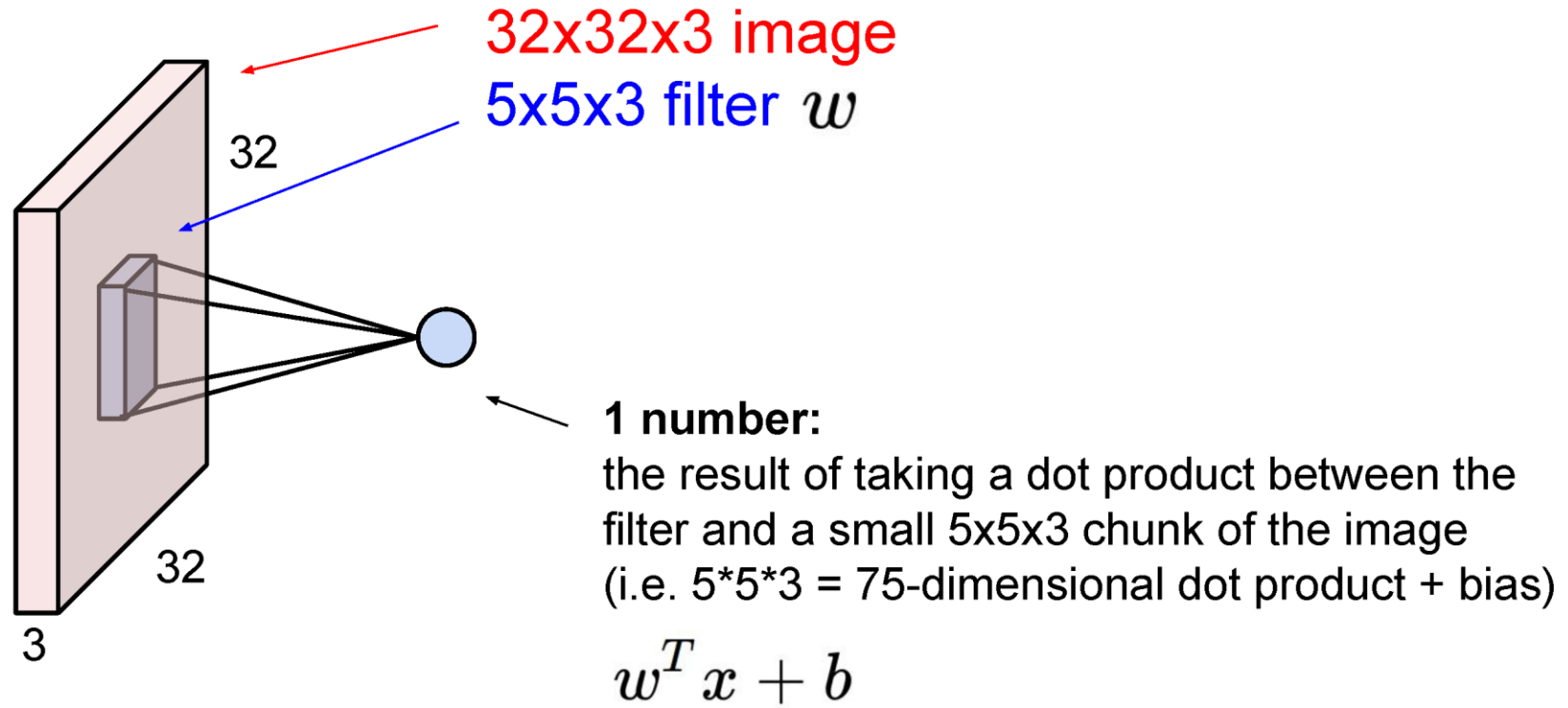
5x5x3 filter



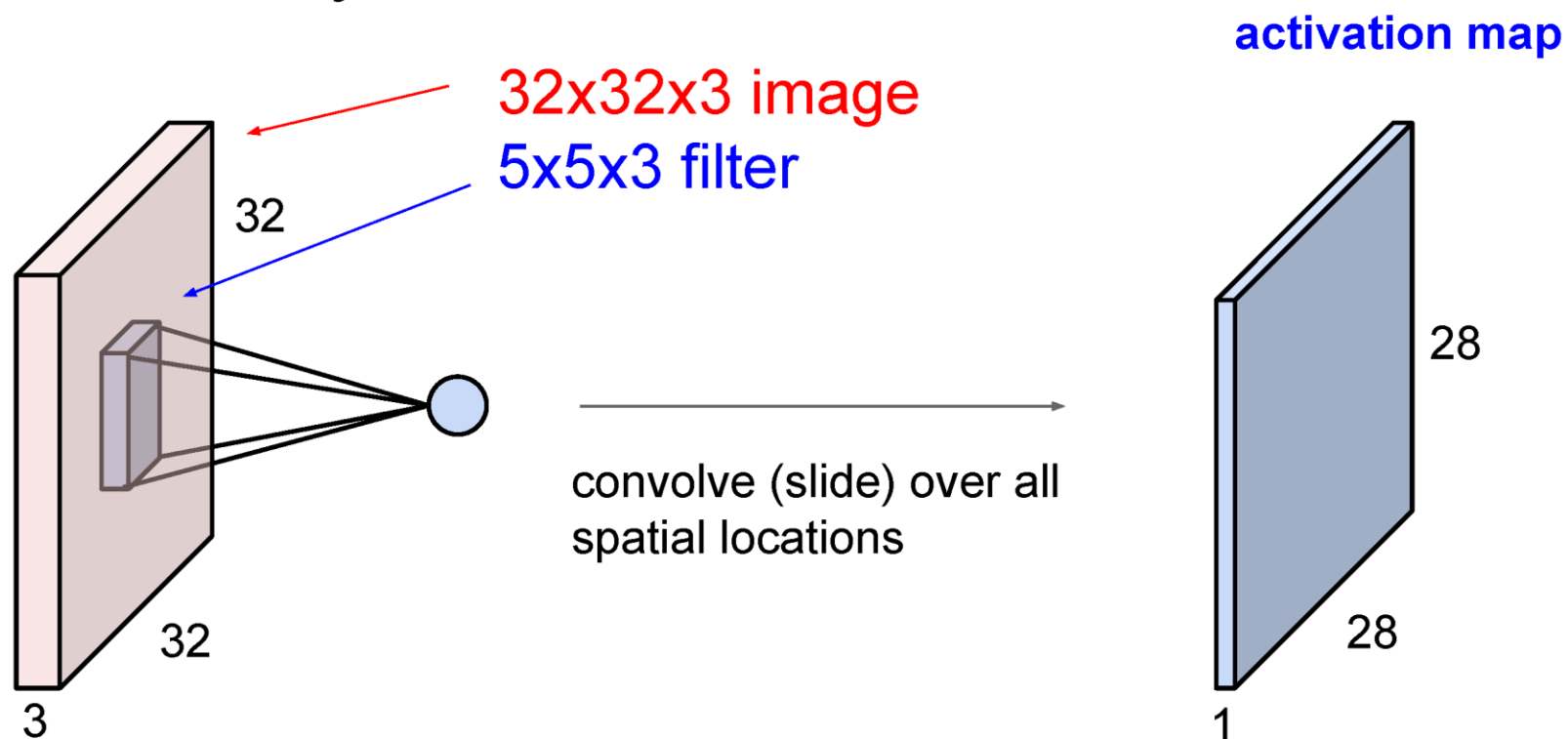
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Number of weights: $5 \times 5 \times 3 + 1 = \mathbf{76}$
(vs. 3072 for a fully-connected layer)
(+1 for bias term)

Convolution Layer

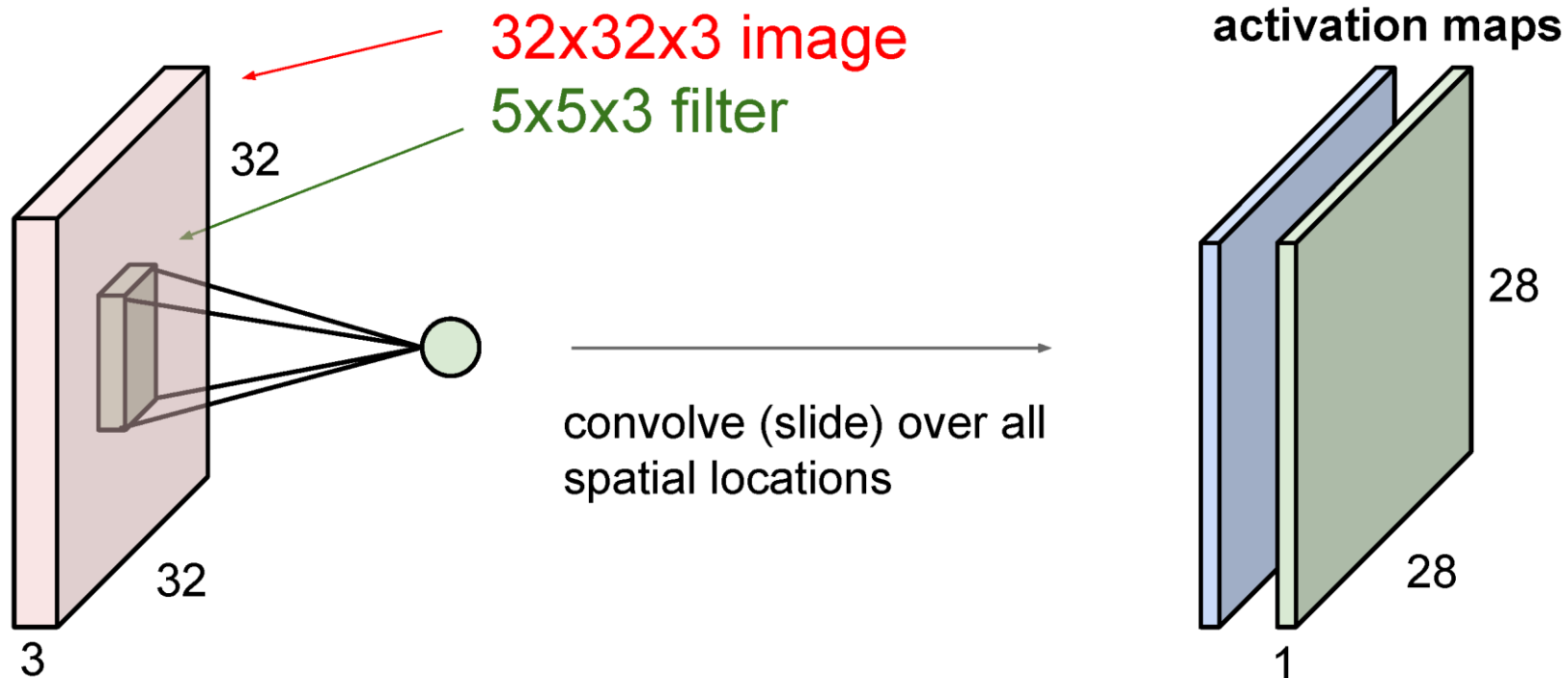


Convolution Layer

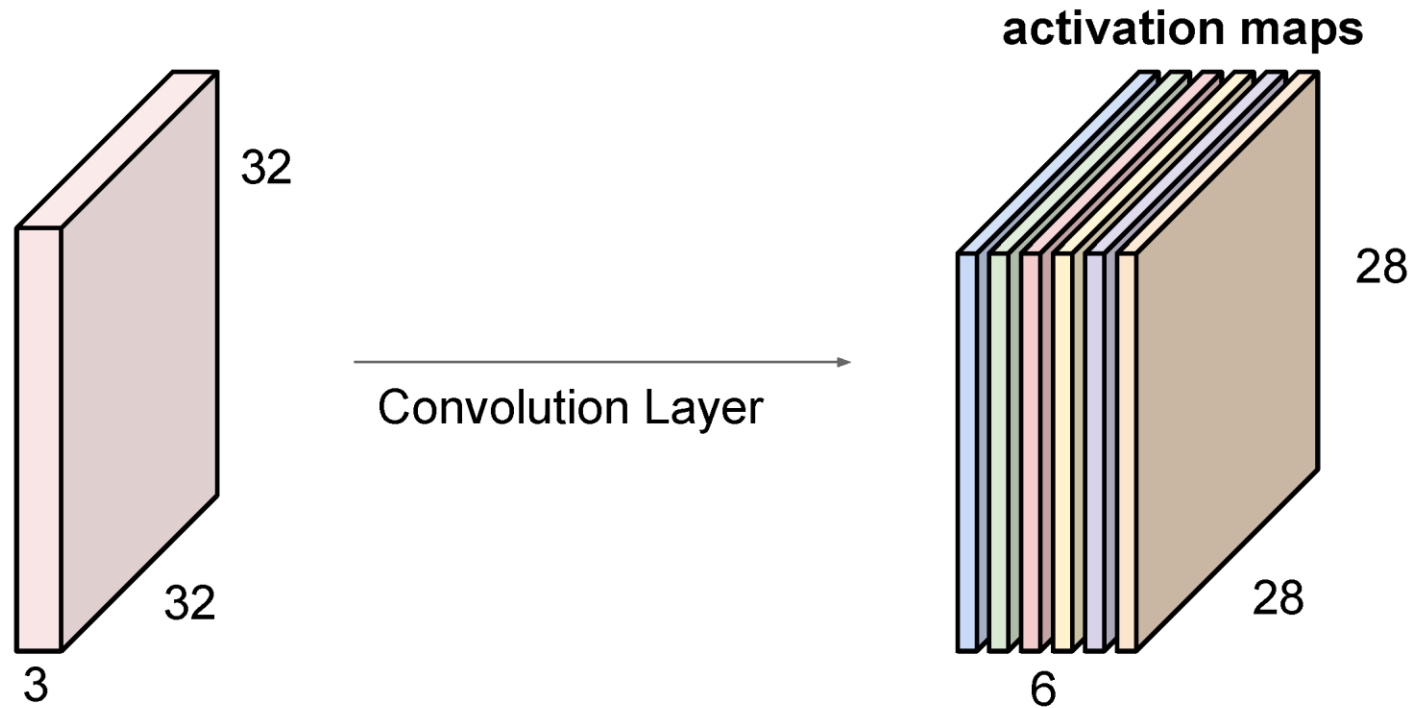


Convolution Layer

consider a second, **green** filter



For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size 28x28x6!

(total number of parameters to learn: $6 \times (75 + 1) = \mathbf{456}$)

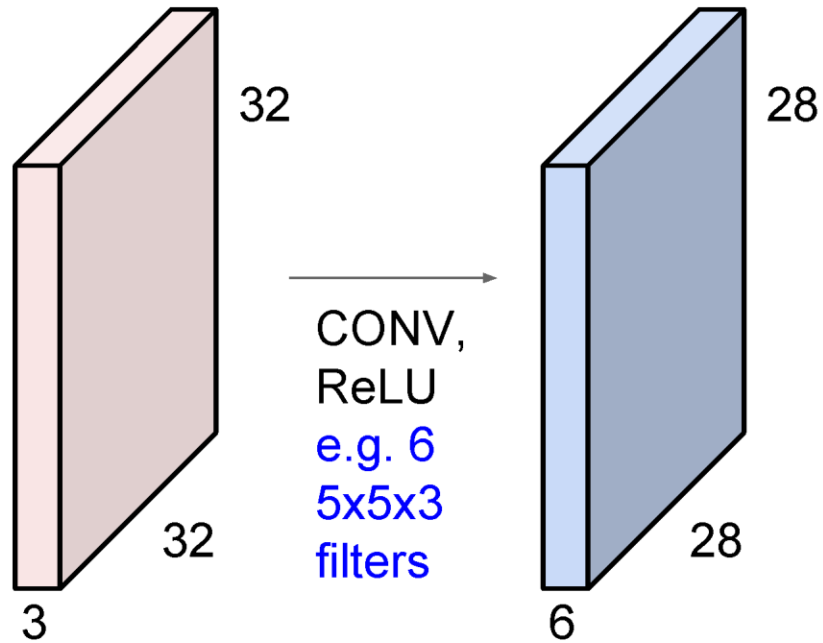
slido



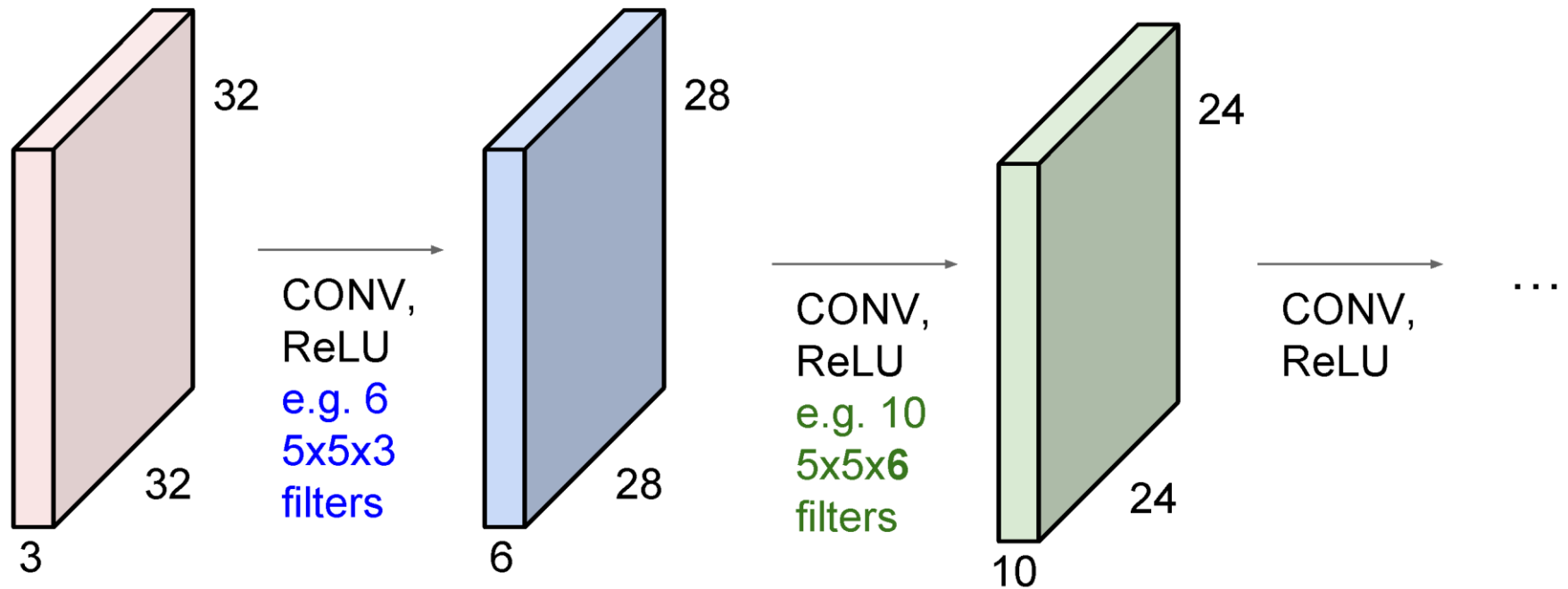
How many parameters are in a convolution layer consisting of 3 $3 \times 3 \times 1$ filters (each with bias term)?

① Start presenting to display the poll results on this slide.

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



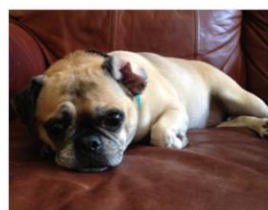
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Preview

[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].

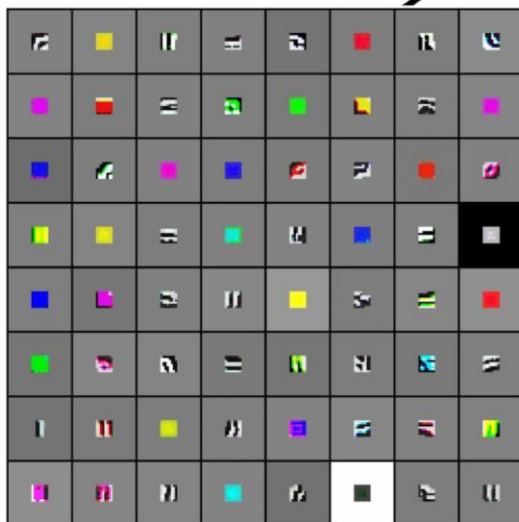


Low-level
features

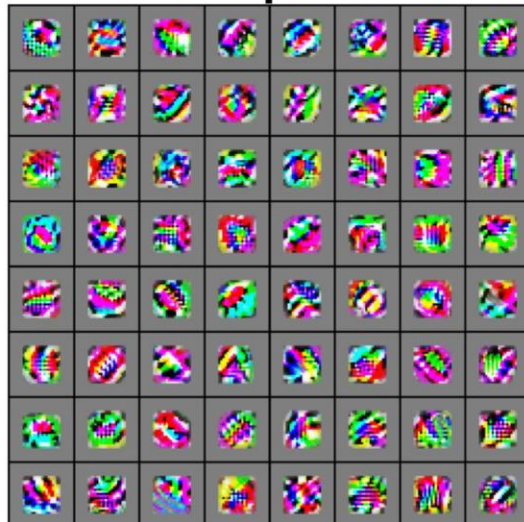
Mid-level
features

High-level
features

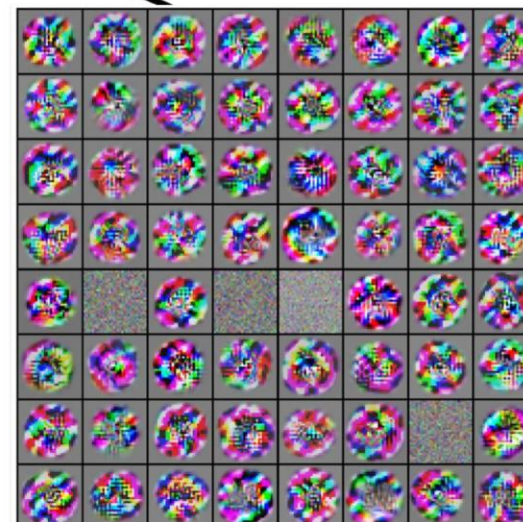
Linearly
separable
classifier



VGG-16 Conv1_1

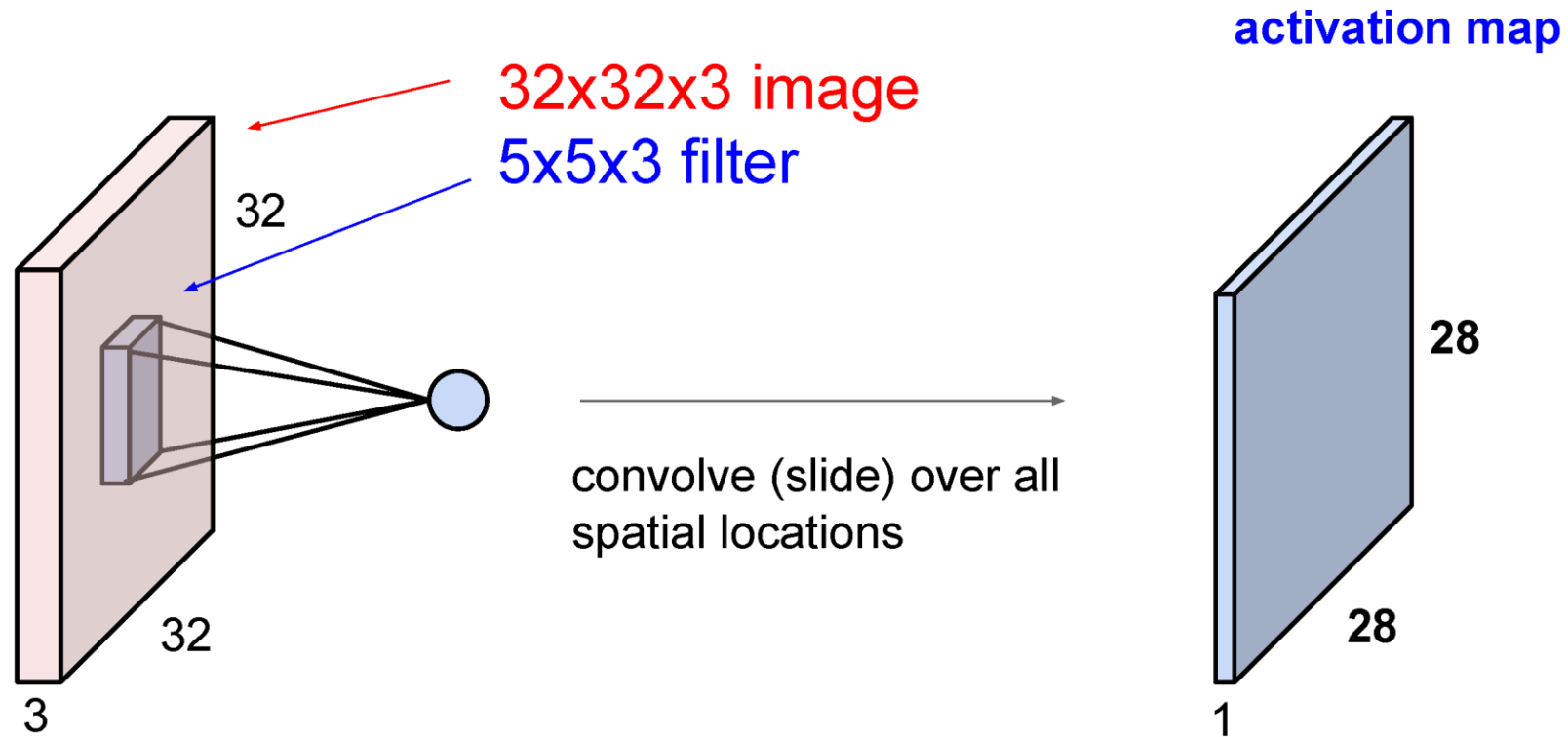


VGG-16 Conv3_2

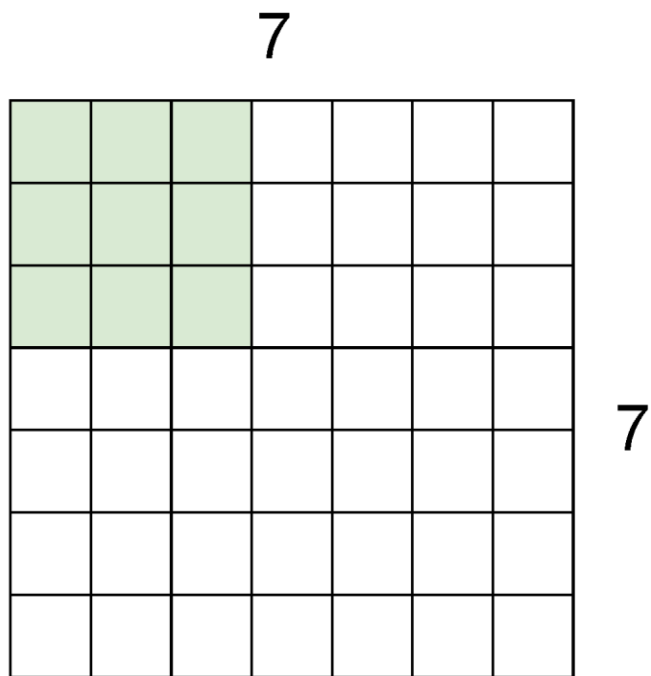


VGG-16 Conv5_3

A closer look at spatial dimensions:

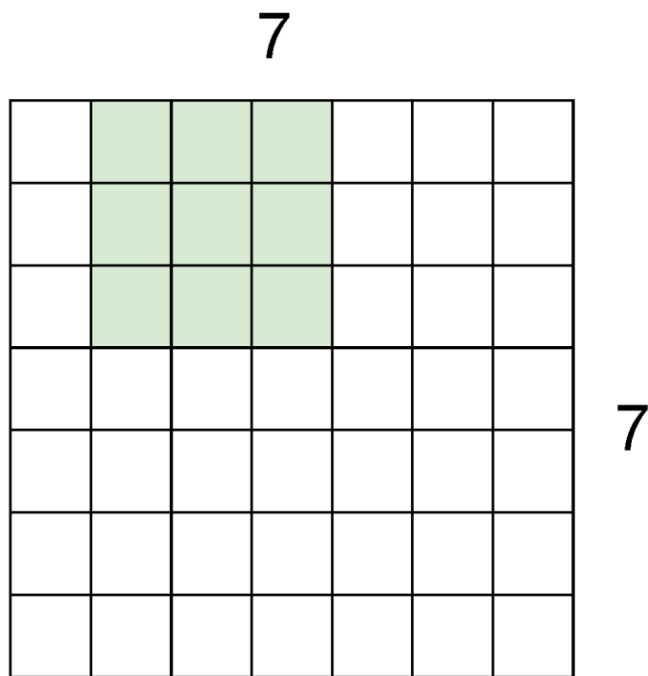


A closer look at spatial dimensions:



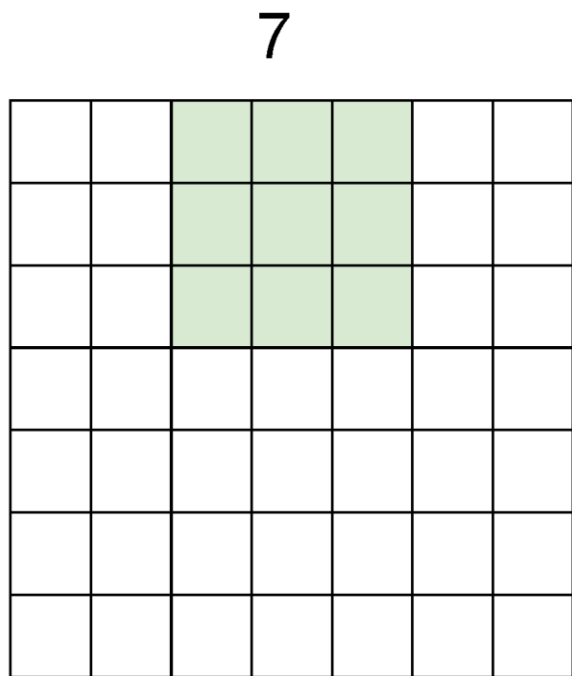
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



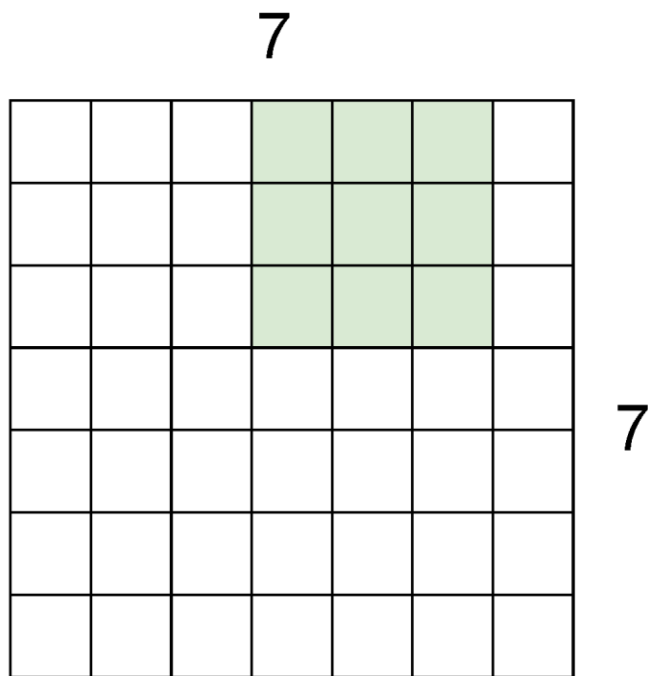
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



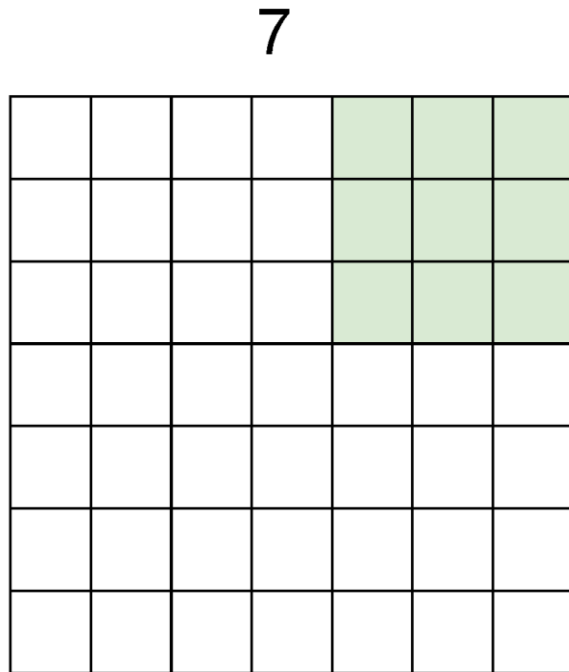
7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

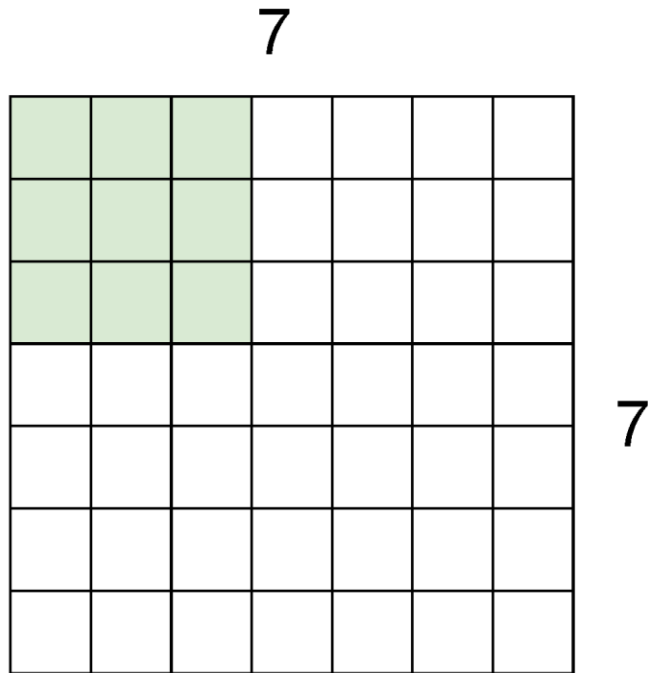
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

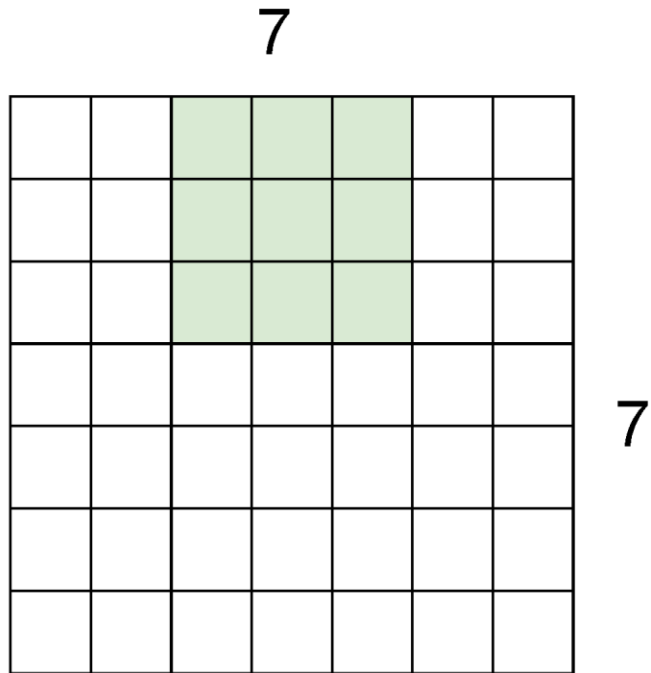
=> 5x5 output

A closer look at spatial dimensions:



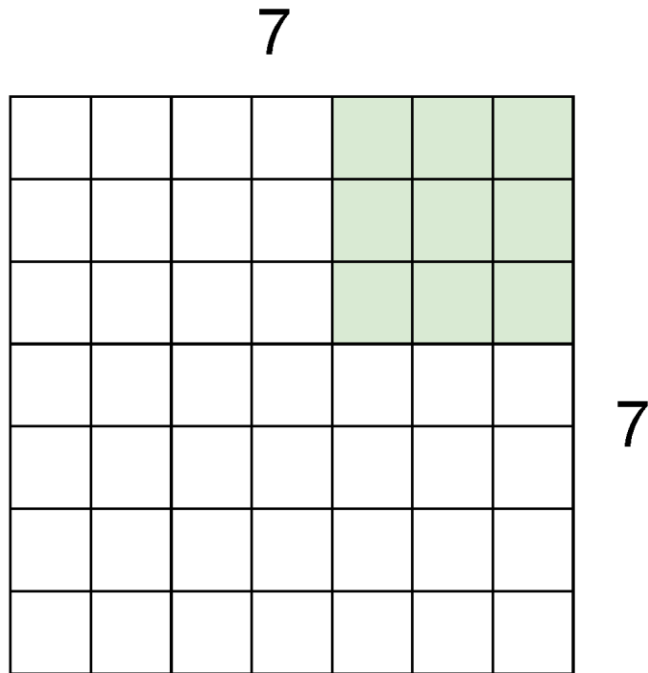
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



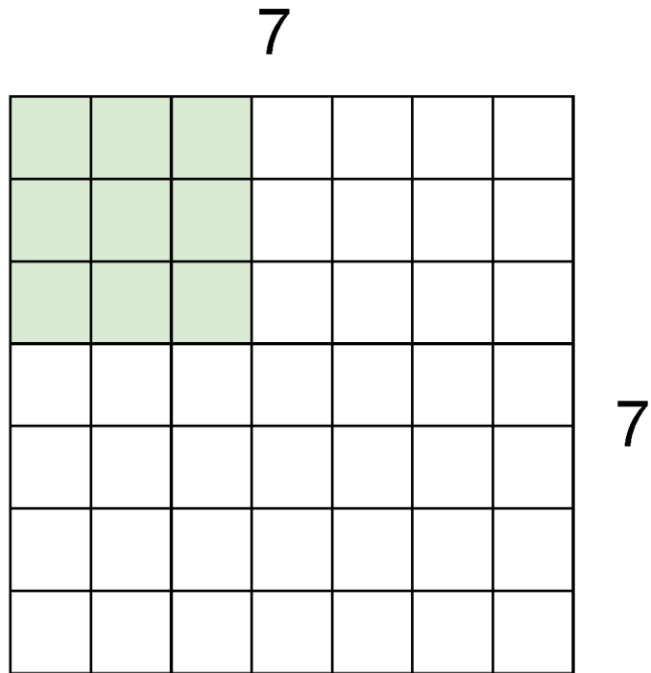
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



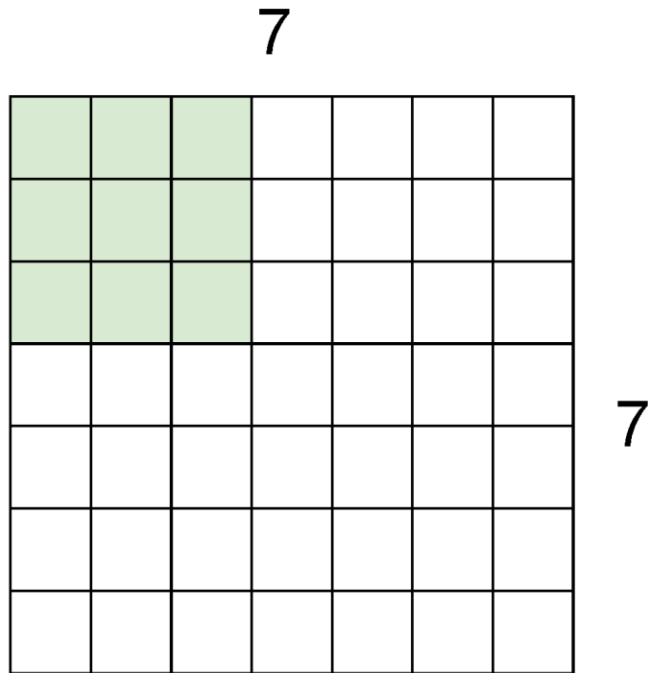
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

A closer look at spatial dimensions:



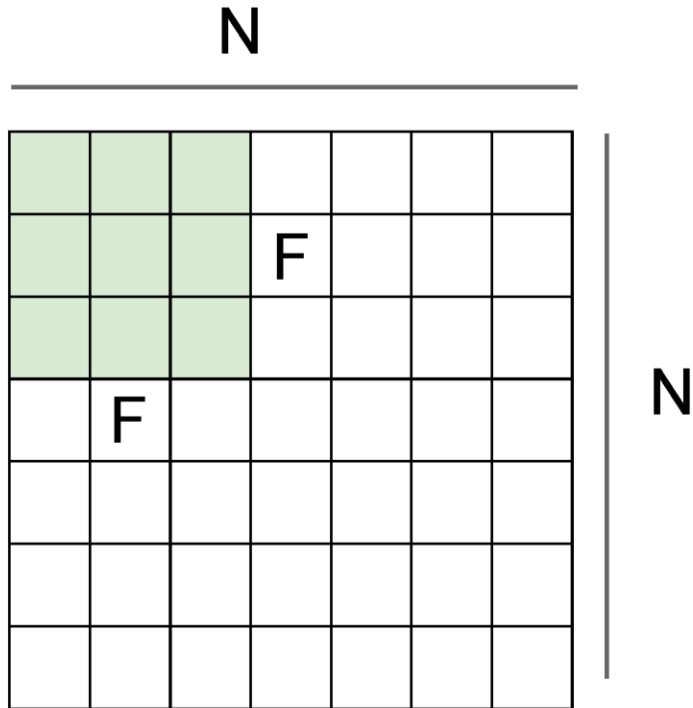
7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.



Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore \backslash$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

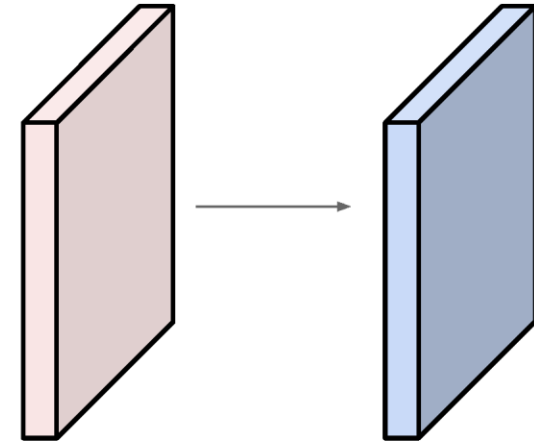
$F = 7 \Rightarrow$ zero pad with 3

Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Output volume size: ?



Examples time:

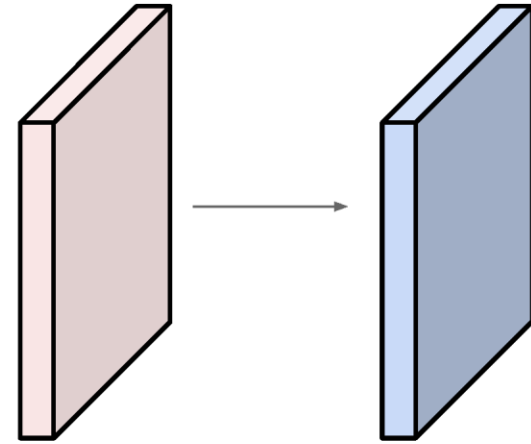
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

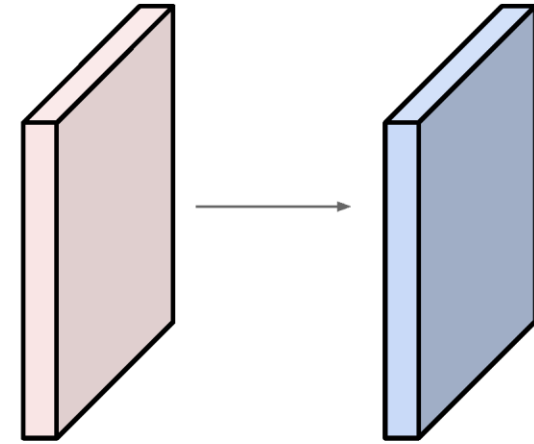
32x32x10



Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

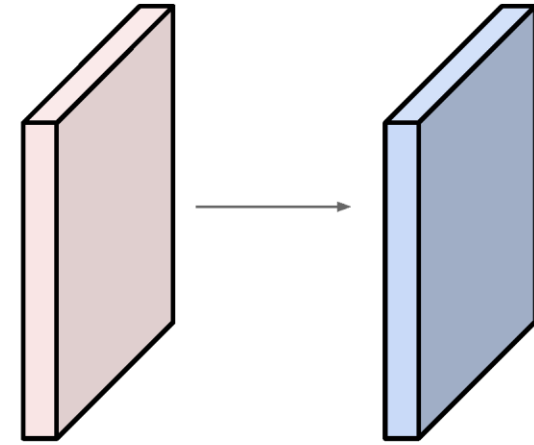


Number of parameters in this layer?

Examples time:

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2

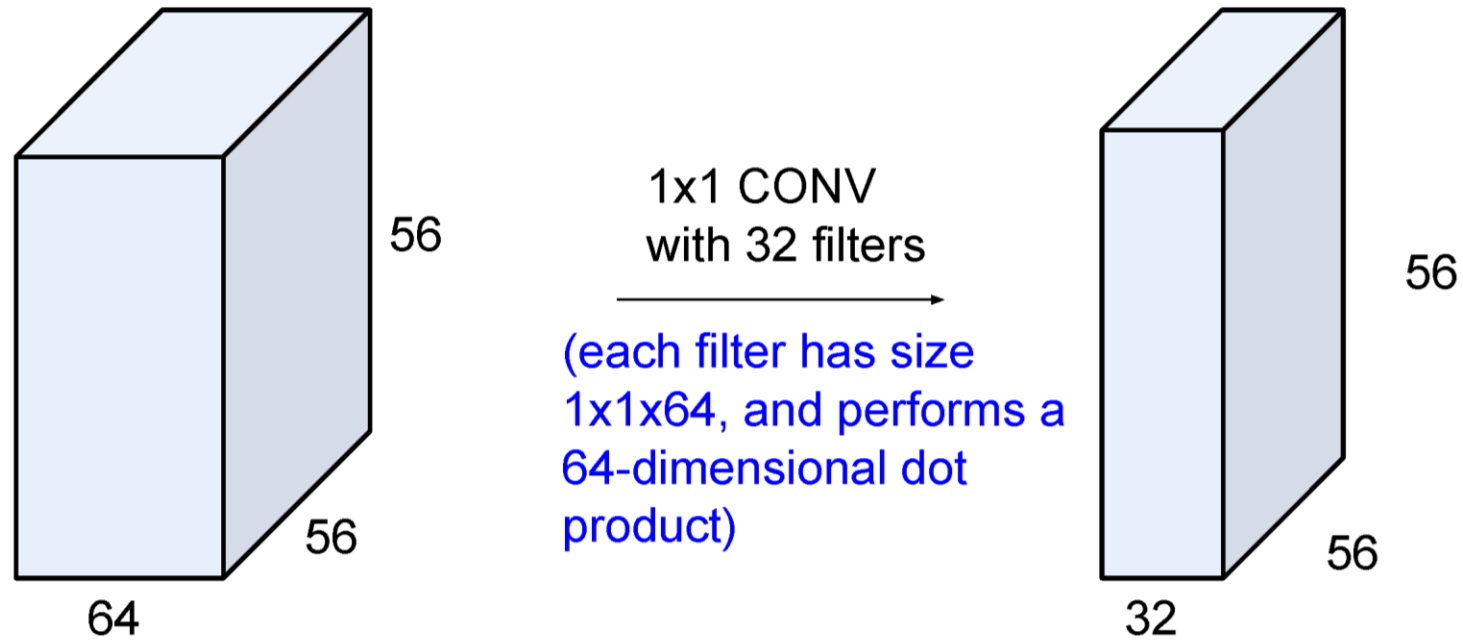


Number of parameters in this layer?

each filter has $5*5*3 + 1 = 76$ params (+1 for bias)

$\Rightarrow 76*10 = 760$

"1x1 convolutions"

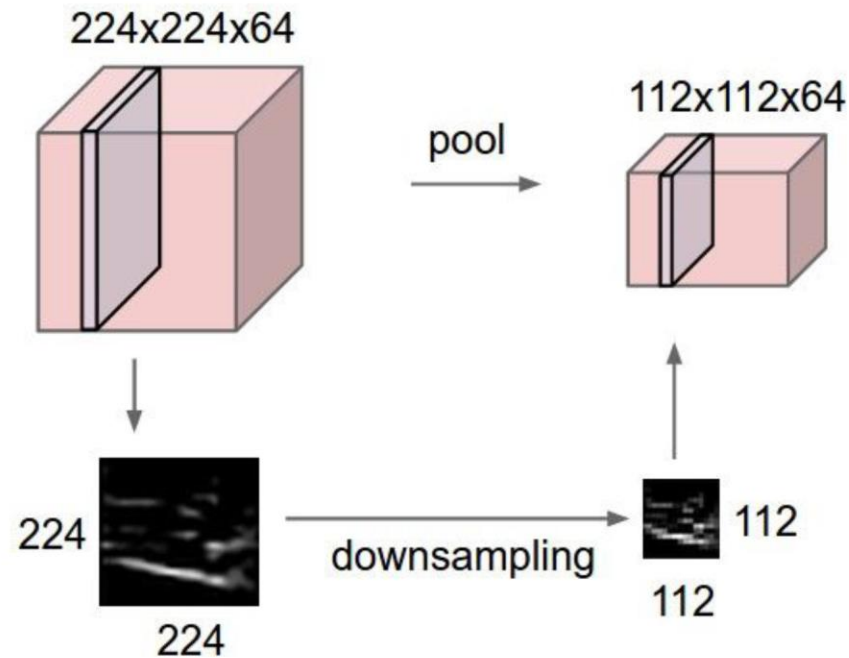


Convolutional layer—properties

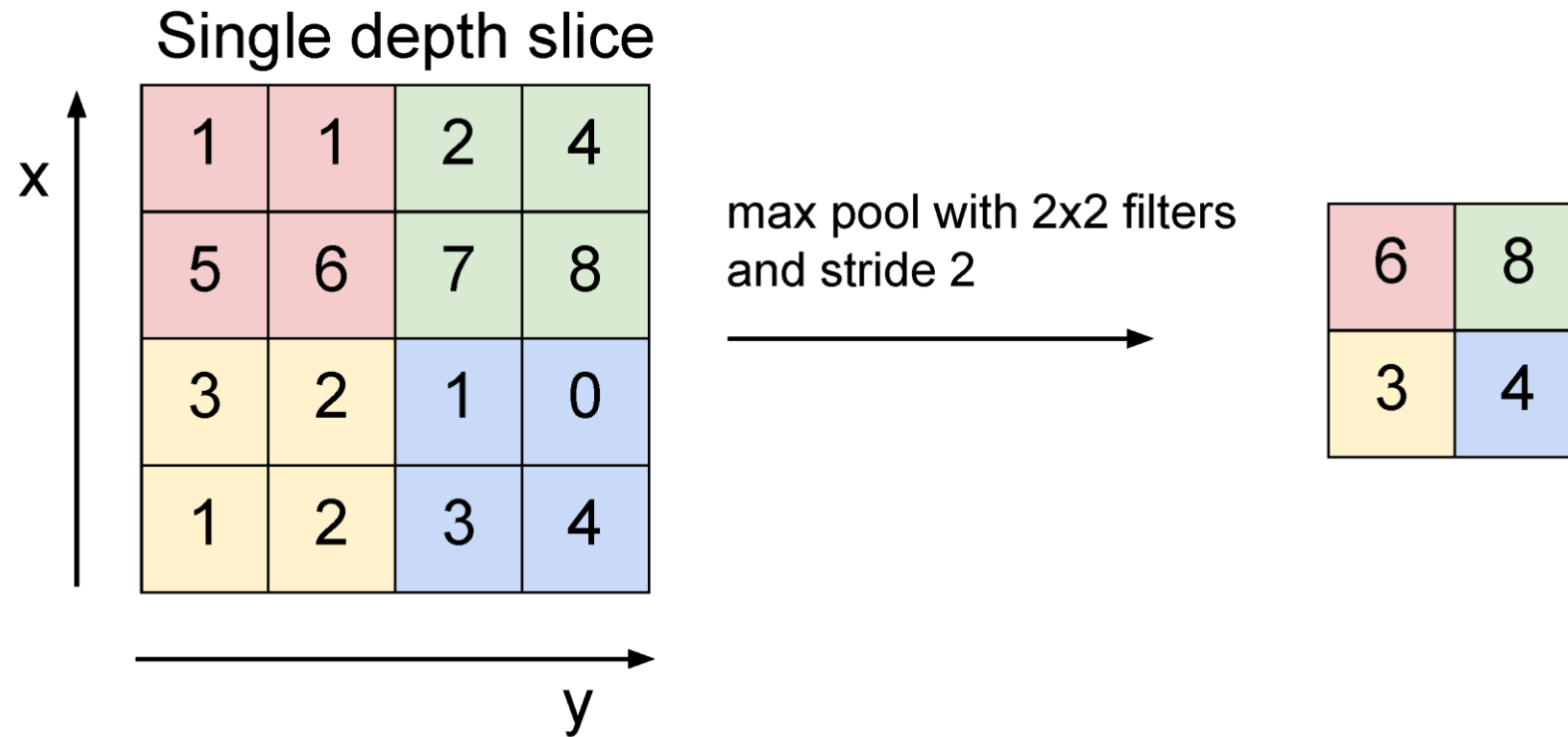
- Small number of parameters to learn compared to a fully connected layer
- Preserves spatial structure—output of a convolutional layer is shaped like an image
- **Translation equivariant:** passing a translated image through a convolutional layer is (almost) equivalent to translating the convolution output (but be careful of image boundaries)

Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:

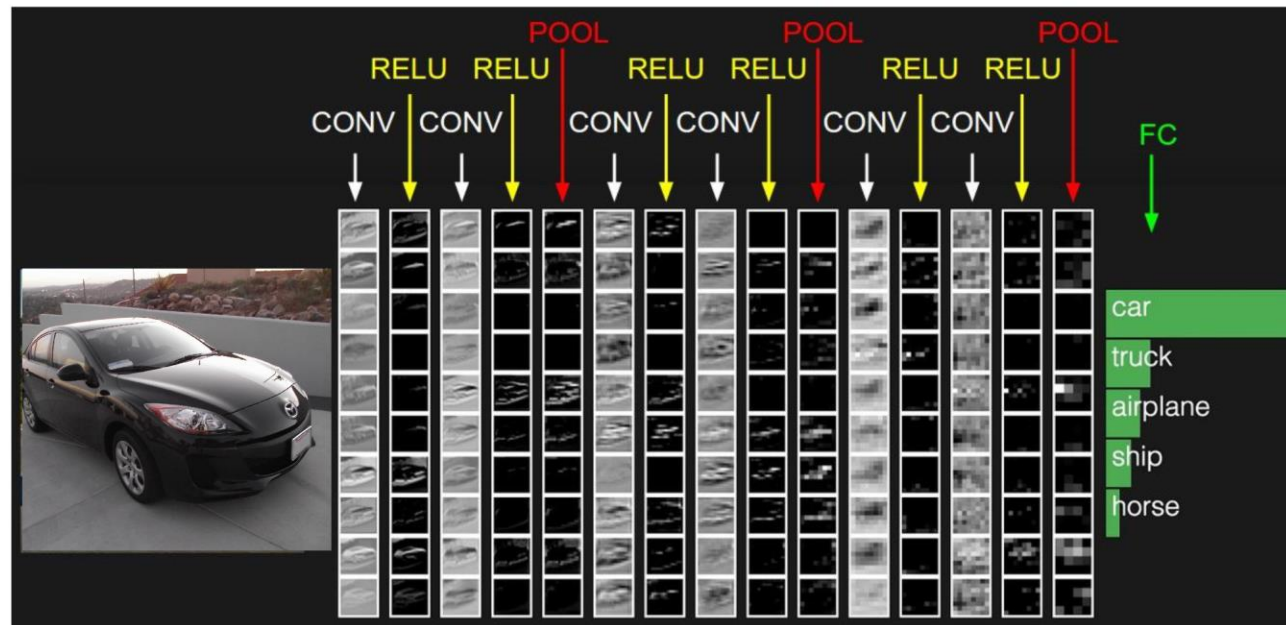


MAX POOLING



Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



[ConvNetJS demo: training on CIFAR-10]

ConvNetJS CIFAR-10 demo

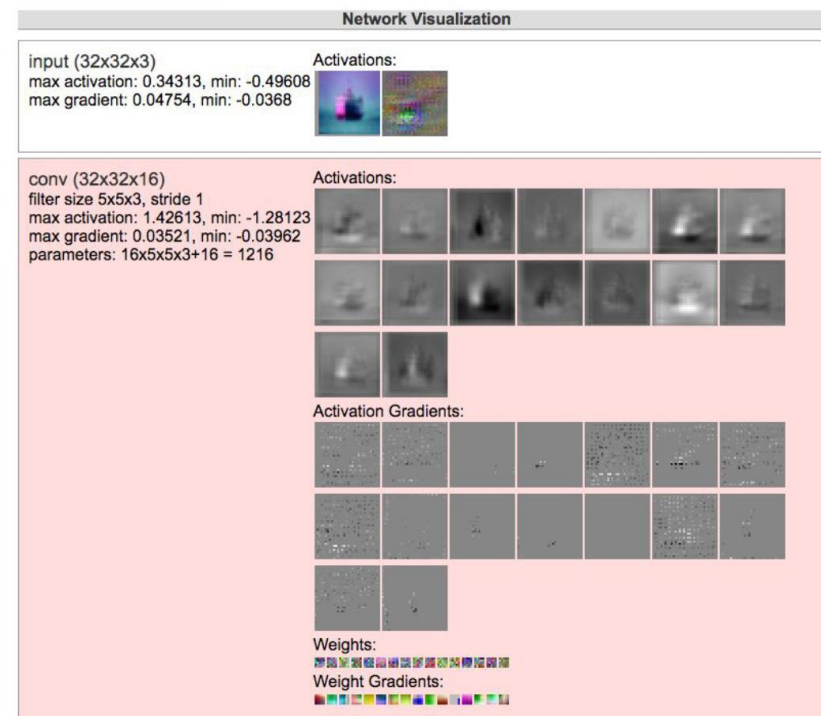
Description

This demo trains a Convolutional Neural Network on the [CIFAR-10 dataset](#) in your browser, with nothing but Javascript. The state of the art on this dataset is about 90% accuracy and human performance is at about 94% (not perfect as the dataset can be a bit ambiguous). I used [this python script](#) to parse the [original files](#) (python version) into batches of images that can be easily loaded into page DOM with img tags.

This dataset is more difficult and it takes longer to train a network. Data augmentation includes random flipping and random image shifts by up to 2px horizontally and vertically.

By default, in this demo we're using Adadelata which is one of per-parameter adaptive step size methods, so we don't have to worry about changing learning rates or momentum over time. However, I still included the text fields for changing these if you'd like to play around with SGD+Momentum trainer.

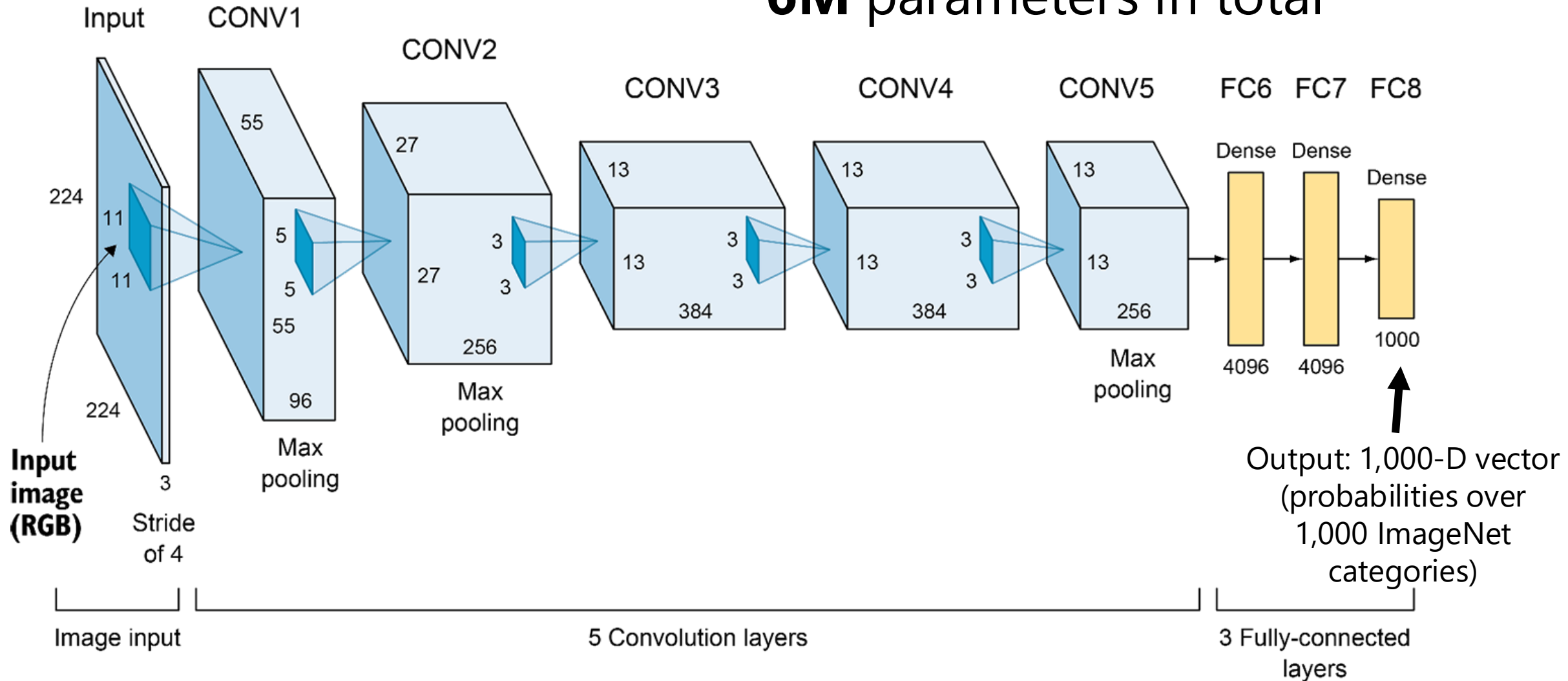
Report questions/bugs/suggestions to [@karpathy](#).



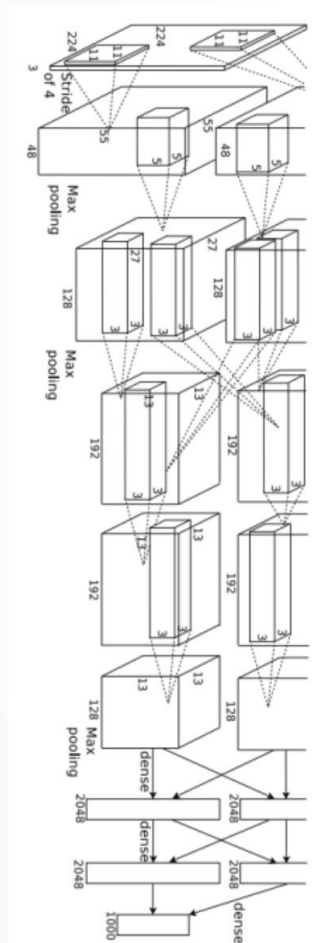
<https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

AlexNet (2012)

6M parameters in total

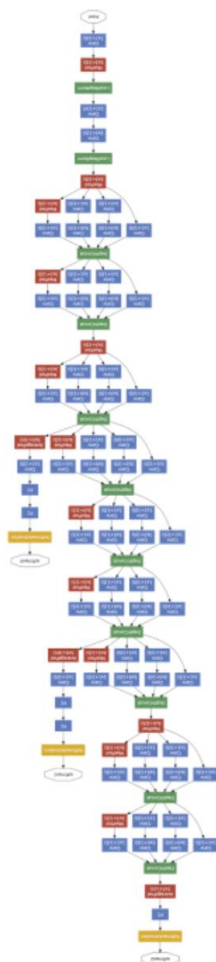


“AlexNet”



[Krizhevsky et al. NIPS 2012]

“GoogLeNet”



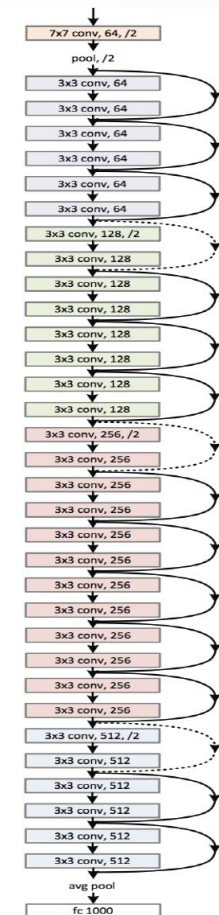
[Szegedy et al. CVPR 2015]

“VGG Net”



[Simonyan & Zisserman, ICLR 2015]

“ResNet”



[He et al. CVPR 2016]

Big picture

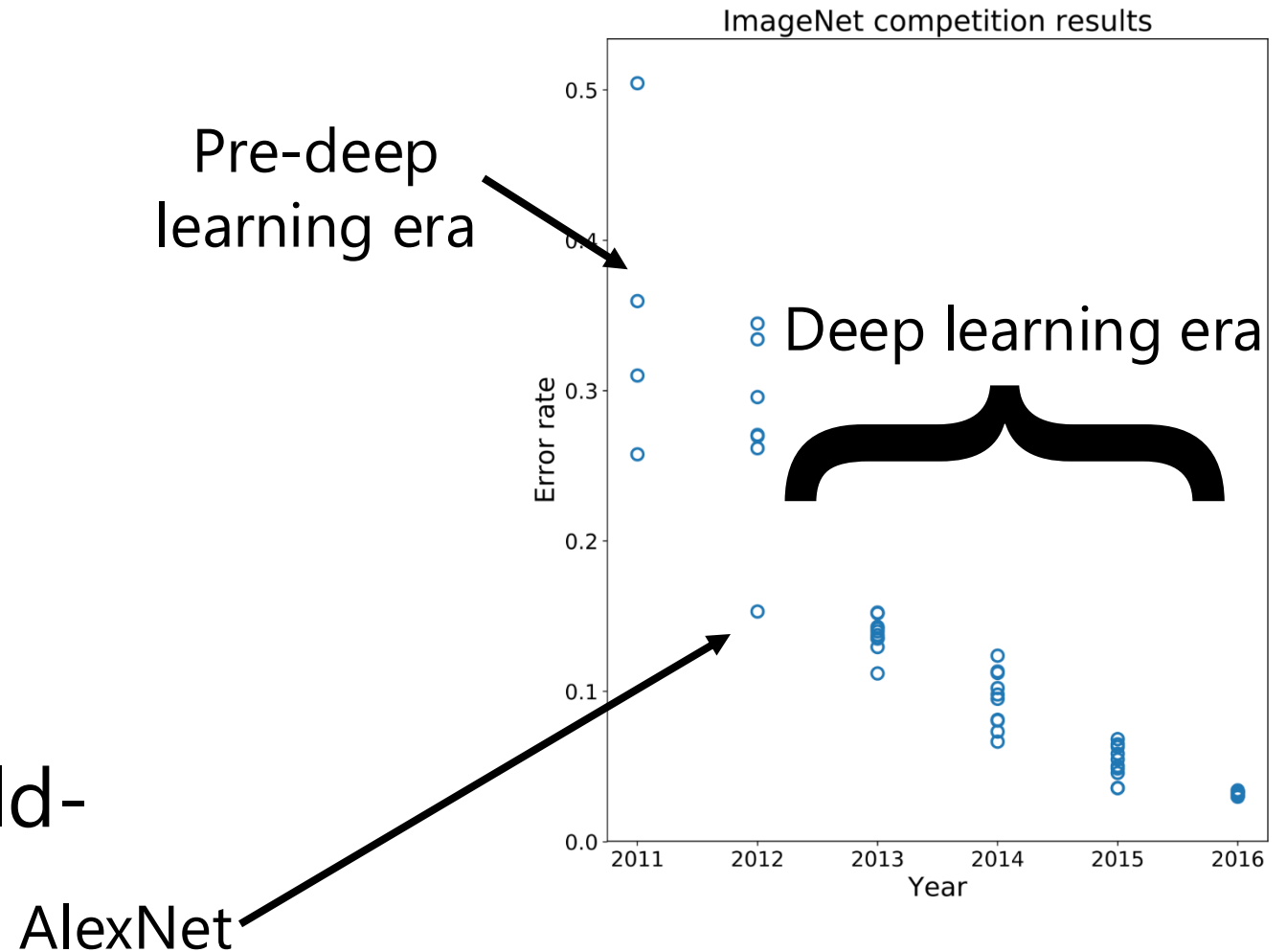
- A convolutional neural network can be thought of as a function from images to class scores
 - With millions of adjustable weights...
 - ... leading to a very non-linear mapping from images to features / class scores.
 - We will set these weights based on classification accuracy on training data...
 - ... and hopefully our network will generalize to new images at test time

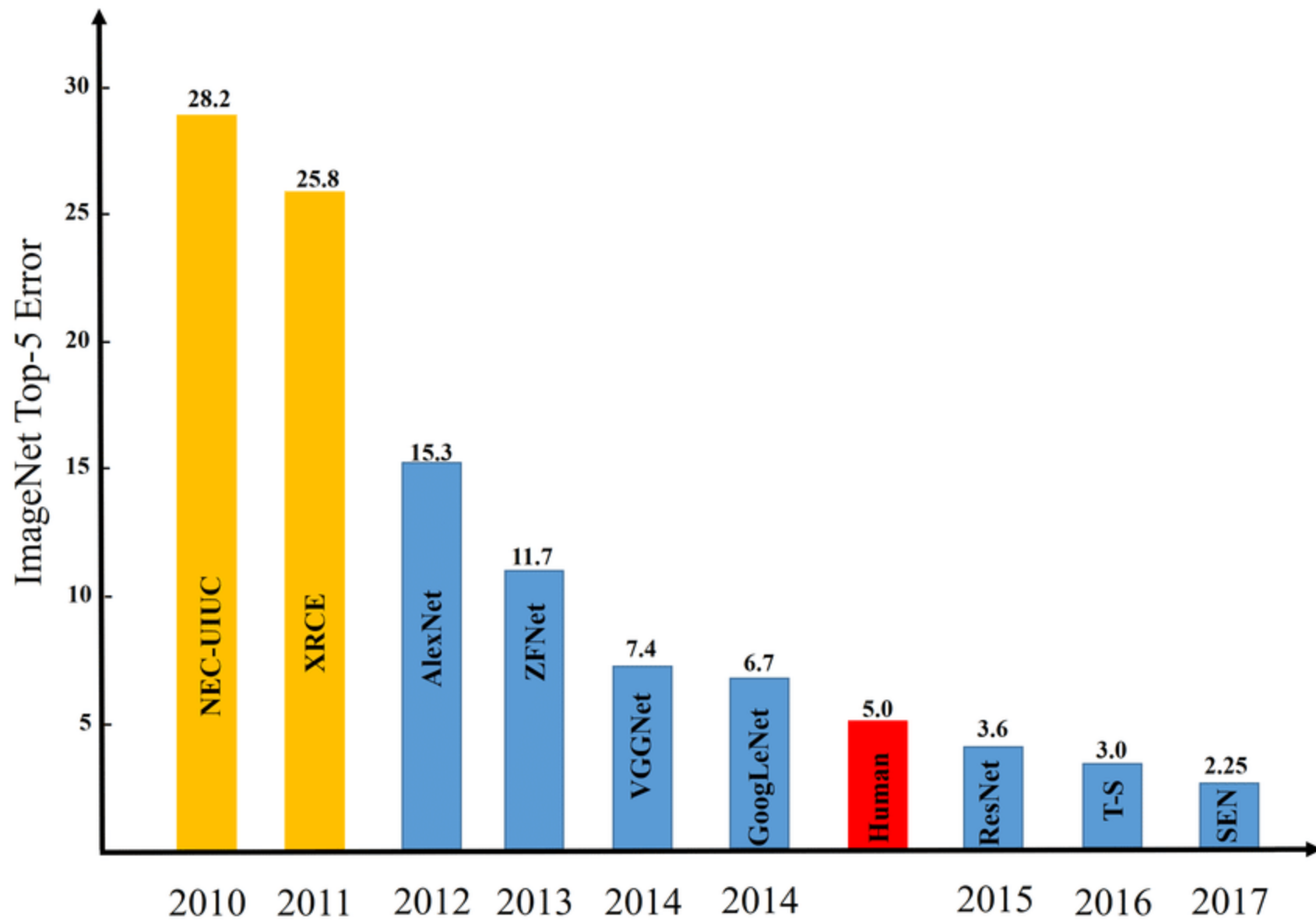
Data is key—enter ImageNet

- ImageNet (and the ImageNet Large-Scale Visual Recognition Challenge, aka **ILSVRC**) has been key to training deep learning methods
 - J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, **ImageNet: A Large-Scale Hierarchical Image Database**. *CVPR*, 2009.
- **ILSVRC**: 1,000 object categories, each with ~700-1300 training images. Test set has 100 images per categories (100,000 total).
- Standard ILSVRC error metric: top-5 error
 - if the correct answer for a given test image is in the top 5 categories, your answer is judged to be correct

Performance improvements on ILSVRC

- ImageNet Large-Scale Visual Recognition Challenge
- Held from 2011-2017
- 1000 categories, 1000 training images per category
- Test performance on held-out test set of images





Questions?